

11 파이썬 모델링 라이브러리





강의 내용

Contents of Lecture

기간	내용	과제
01주차	오리엔테이션 Python 언어의 기본, lpython, 주피터 노트북	-
02주차	내장 자료구조, 함수, 파일	실습 과제
03주차	Numpy 기본 : 배열과 벡터 연산	실습 과제
04주차	Pandas 시작하기	실습 과제
05주차	데이터 로딩과 저장, 파일 형식	실습 과제
06주차	데이터 정제 및 준비(조인, 병합, 변형)	실습 과제
07주차	그래프와 시각화	기말 프로젝트 상세 공지
08주차	중간고사 - 오프라인 오픈북으로 진행	실습 과제

기간	내용	과제
09주차	데이터 집계와 그룹 연산	실습 과제
10주차	시계열	실습 과제
11주차	고급 pandas	실습 과제
12주차	파이썬 모델링 라이브러리	실습 과제
13주차	데이터 분석 예제 / 고급 Numpy	실습 과제
14주차	Kaggle / Dacon 데이터 분석 사례 Case Study	프로젝트 레포트 제출
15주차	[기말고사] Team별 프로젝트 결과 발표	종강~!

1부: 파이썬 모델링 라이브러리

- 1) pandas와 모델 코드 인터페이싱
- 2) patsy를 이용한 모델 생성
- 3) statsmodels 소개
- 4) scikit-learn 소개





Python 모델링 라이브러리

Introduction to Modeling Libraries in Python

- 수업 내용은 Python에서 데이터 분석을 수행하기 위한 프로그래밍 기반을 제공하는 데 중점을 둠
- 데이터 분석가와 과학자들은 종종 데이터 랭글링과 준비에 과도하게 많은 시간을 소비한다고 보고, 따라서 이 수업의 구조는 이러한 기술을 숙달하는 것의 중요성 반영
- 많은 통계 문제는 일반 최소 제곱 회귀와 같은 간단한 기술로 해결할 수 있지만, 다른 문제는 고급 기계 학습 방법이 필요할 수 있음
- 다행히도 Python은 분석 방법을 구현하기 위해 선택하는 언어 중 하나가 되었기 때문에 이 수업을 마친 후에 탐색할 수 있는 많은 도구가 있음
- 이번 주 수업에서는 pandas로 데이터 랭글링과 모델 피팅 및 채점 사이클을 오갈 때 도움이 될 수 있는 pandas의 몇 가지 기능 검토
- 다음 두 가지 인기 있는 모델링 도구인 statsmodels 및 scikit-learn에 대해 간략히 소개
 - 각 프로젝트는 자체 전용 책을 보증할 만큼 충분히 크기 때문에 포괄적인 노력을 기울이지 않고, 대신 데이터 과학, 통계 및 기계 학습에 대한 다른 Python 기반 책과 함께 두 프로젝트의 온라인 문서로 안내



pandas와 모델 코드의 인터페이스

Interfacing Between pandas and Model Code

13.1 Interfacing Between pandas and Model Code

A common workflow for model development is to use pandas for data loading and cleaning before switching over to a modeling library to build the model itself. An important part of the model development process is called *feature engineering* in machine learning. This can describe any data transformation or analytics that extract information from a raw dataset that may be useful in a modeling context. The data aggregation and GroupBy tools we have explored in this book are used often in a feature engineering context.

While details of “good” feature engineering are out of scope for this book, I will show some methods to make switching between data manipulation with pandas and modeling as painless as possible.

The point of contact between pandas and other analysis libraries is usually NumPy arrays. To turn a DataFrame into a NumPy array, use the `.values` property:

```
In [10]: import pandas as pd
```

```
In [11]: import numpy as np
```

```
In [12]: data = pd.DataFrame({
.....:     'x0': [1, 2, 3, 4, 5],
.....:     'x1': [0.01, -0.01, 0.25, -4.1, 0.],
.....:     'y': [-1.5, 0., 3.6, 1.3, -2.]})
```

```
In [13]: data
Out[13]:
   x0  x1  y
0   1  0.01 -1.5
1   2 -0.01  0.0
2   3  0.25  3.6
3   4 -4.10  1.3
4   5  0.00 -2.0
```

```
In [14]: data.columns
Out[14]: Index(['x0', 'x1', 'y'], dtype='object')
```

```
In [15]: data.values
Out[15]:
array([[ 1. ,  0.01, -1.5 ],
       [ 2. , -0.01,  0. ],
       [ 3. ,  0.25,  3.6 ],
       [ 4. , -4.1 ,  1.3 ],
       [ 5. ,  0. , -2. ]])
```

To convert back to a DataFrame, as you may recall from earlier chapters, you can pass a two-dimensional ndarray with optional column names:

```
In [16]: df2 = pd.DataFrame(data.values, columns=['one', 'two', 'three'])
```

```
In [17]: df2
Out[17]:
   one  two  three
0  1.0  0.01  -1.5
1  2.0 -0.01   0.0
2  3.0  0.25   3.6
3  4.0 -4.10   1.3
4  5.0  0.00  -2.0
```



The `.values` attribute is intended to be used when your data is homogeneous—for example, all numeric types. If you have heterogeneous data, the result will be an ndarray of Python objects:

```
In [18]: df3 = data.copy()
```

```
In [19]: df3['strings'] = ['a', 'b', 'c', 'd', 'e']
```

```
In [20]: df3
Out[20]:
   x0  x1  y strings
0   1  0.01 -1.5     a
1   2 -0.01  0.0     b
2   3  0.25  3.6     c
3   4 -4.10  1.3     d
4   5  0.00 -2.0     e
```

```
In [21]: df3.values
Out[21]:
array([[1, 0.01, -1.5, 'a'],
       [2, -0.01, 0.0, 'b'],
       [3, 0.25, 3.6, 'c'],
       [4, -4.1, 1.3, 'd'],
       [5, 0.0, -2.0, 'e']], dtype=object)
```




pandas와 모델 코드의 인터페이스

Interfacing Between pandas and Model Code

For some models, you may only wish to use a subset of the columns. I recommend using loc indexing with values:

```
In [22]: model_cols = ['x0', 'x1']

In [23]: data.loc[:, model_cols].values
Out[23]:
array([[ 1. ,  0.01],
       [ 2. , -0.01],
       [ 3. ,  0.25],
       [ 4. , -4.1 ],
       [ 5. ,  0.  ]])
```

Some libraries have native support for pandas and do some of this work for you automatically: converting to NumPy from DataFrame and attaching model parameter names to the columns of output tables or Series. In other cases, you will have to perform this “metadata management” manually.

In [Chapter 12](#) we looked at pandas’s Categorical type and the pandas.get_dummies function. Suppose we had a non-numeric column in our example dataset:

```
In [24]: data['category'] = pd.Categorical(['a', 'b', 'a', 'a', 'b'],
.....:                                     categories=['a', 'b'])
```

```
In [25]: data
Out[25]:
```

	x0	x1	y	category
0	1	0.01	-1.5	a
1	2	-0.01	0.0	b
2	3	0.25	3.6	a
3	4	-4.10	1.3	a
4	5	0.00	-2.0	b

If we wanted to replace the 'category' column with dummy variables, we create dummy variables, drop the 'category' column, and then join the result:

```
In [26]: dummies = pd.get_dummies(data.category, prefix='category')

In [27]: data_with_dummies = data.drop('category', axis=1).join(dummies)

In [28]: data_with_dummies
Out[28]:
```

	x0	x1	y	category_a	category_b
0	1	0.01	-1.5	1	0
1	2	-0.01	0.0	0	1
2	3	0.25	3.6	1	0
3	4	-4.10	1.3	1	0
4	5	0.00	-2.0	0	1

There are some nuances to fitting certain statistical models with dummy variables. It may be simpler and less error-prone to use Patsy (the subject of the next section) when you have more than simple numeric columns.



Patsy를 이용해서 모델 생성하기

Creating Model Descriptions with Patsy

13.2 Creating Model Descriptions with Patsy

Patsy is a Python library for describing statistical models (especially linear models) with a small string-based “formula syntax,” which is inspired by (but not exactly the same as) the formula syntax used by the R and S statistical programming languages.

Patsy is well supported for specifying linear models in statsmodels, so I will focus on some of the main features to help you get up and running. Patsy’s *formulas* are a special string syntax that looks like:

```
y ~ x0 + x1
```

The syntax `a + b` does not mean to add `a` to `b`, but rather that these are *terms* in the *design matrix* created for the model. The `patsy.dmatrices` function takes a formula string along with a dataset (which can be a `DataFrame` or a dict of arrays) and produces design matrices for a linear model:

```
In [29]: data = pd.DataFrame({
.....:     'x0': [1, 2, 3, 4, 5],
.....:     'x1': [0.01, -0.01, 0.25, -4.1, 0.],
.....:     'y': [-1.5, 0., 3.6, 1.3, -2.]})
```

```
In [30]: data
Out[30]:
   x0  x1  y
0   1  0.01 -1.5
1   2 -0.01  0.0
2   3  0.25  3.6
3   4 -4.10  1.3
4   5  0.00 -2.0
```

```
In [31]: import patsy
```

```
In [32]: y, X = patsy.dmatrices('y ~ x0 + x1', data)
```

Now we have:

```
In [33]: y
Out[33]:
DesignMatrix with shape (5, 1)
   y
-1.5
 0.0
 3.6
 1.3
-2.0
Terms:
   'y' (column 0)
```

```
In [34]: X
Out[34]:
DesignMatrix with shape (5, 3)
Intercept  x0  x1
   1   1  0.01
   1   2 -0.01
   1   3  0.25
   1   4 -4.10
   1   5  0.00
Terms:
   'Intercept' (column 0)
   'x0' (column 1)
   'x1' (column 2)
```



Patsy를 이용해서 모델 생성하기

Creating Model Descriptions with Patsy

These Patsy DesignMatrix instances are NumPy ndarrays with additional metadata:

```
In [35]: np.asarray(y)
Out[35]:
array([[ -1.5],
       [  0. ],
       [  3.6],
       [  1.3],
       [ -2. ]])

In [36]: np.asarray(X)
Out[36]:
array([[ 1. ,  1. ,  0.01],
       [ 1. ,  2. , -0.01],
       [ 1. ,  3. ,  0.25],
       [ 1. ,  4. , -4.1 ],
       [ 1. ,  5. ,  0. ]])
```

You might wonder where the Intercept term came from. This is a convention for linear models like ordinary least squares (OLS) regression. You can suppress the intercept by adding the term + 0 to the model:

```
In [37]: patsy.dmatrices('y ~ x0 + x1 + 0', data)[1]
Out[37]:
DesignMatrix with shape (5, 2)
   x0    x1
1  0.01
2 -0.01
3  0.25
4 -4.10
5  0.00
Terms:
  'x0' (column 0)
  'x1' (column 1)
```

The Patsy objects can be passed directly into algorithms like `numpy.linalg.lstsq`, which performs an ordinary least squares regression:

```
In [38]: coef, resid, _, _ = np.linalg.lstsq(X, y)
```

The model metadata is retained in the `design_info` attribute, so you can reattach the model column names to the fitted coefficients to obtain a Series, for example:

```
In [39]: coef
Out[39]:
array([[ 0.3129],
       [-0.0791],
       [-0.2655]])

In [40]: coef = pd.Series(coef.squeeze(), index=X.design_info.column_names)

In [41]: coef
Out[41]:
Intercept    0.312910
x0          -0.079106
x1          -0.265464
dtype: float64
```




Patsy를 이용해서 모델 생성하기 > Patsy 용법으로 데이터 변환하기

Creating Model Descriptions with Patsy

Data Transformations in Patsy Formulas

You can mix Python code into your Patsy formulas; when evaluating the formula the library will try to find the functions you use in the enclosing scope:

```
In [42]: y, X = patsy.dmatrices('y ~ x0 + np.log(np.abs(x1) + 1)', data)
```

```
In [43]: X
```

```
Out[43]:
```

```
DesignMatrix with shape (5, 3)
```

	Intercept	x0	np.log(np.abs(x1) + 1)
1	1	0.00995	
1	2	0.00995	
1	3	0.22314	
1	4	1.62924	
1	5	0.00000	

```
Terms:
```

```
'Intercept' (column 0)
'x0' (column 1)
'np.log(np.abs(x1) + 1)' (column 2)
```

Some commonly used variable transformations include standardizing (to mean 0 and variance 1) and centering (subtracting the mean). Patsy has built-in functions for this purpose:

```
In [44]: y, X = patsy.dmatrices('y ~ standardize(x0) + center(x1)', data)
```

```
In [45]: X
```

```
Out[45]:
```

```
DesignMatrix with shape (5, 3)
```

	Intercept	standardize(x0)	center(x1)
1	-1.41421	0.78	
1	-0.70711	0.76	
1	0.00000	1.02	
1	0.70711	-3.33	
1	1.41421	0.77	

```
Terms:
```

```
'Intercept' (column 0)
'standardize(x0)' (column 1)
'center(x1)' (column 2)
```

As part of a modeling process, you may fit a model on one dataset, then evaluate the model based on another. This might be a *hold-out* portion or new data that is observed later. When applying transformations like center and standardize, you should be careful when using the model to form predications based on new data. These are called *stateful* transformations, because you must use statistics like the mean or standard deviation of the original dataset when transforming a new dataset.

The `patsy.build_design_matrices` function can apply transformations to new *out-of-sample* data using the saved information from the original *in-sample* dataset:

```
In [46]: new_data = pd.DataFrame([
.....:     'x0': [6, 7, 8, 9],
.....:     'x1': [3.1, -0.5, 0, 2.3],
.....:     'y': [1, 2, 3, 4]])
```

```
In [47]: new_X = patsy.build_design_matrices([X.design_info], new_data)
```

```
In [48]: new_X
```

```
Out[48]:
```

```
[DesignMatrix with shape (4, 3)
```

	Intercept	standardize(x0)	center(x1)
1	2.12132	3.87	
1	2.82843	0.27	
1	3.53553	0.77	
1	4.24264	3.07	

```
Terms:
```

```
'Intercept' (column 0)
'standardize(x0)' (column 1)
'center(x1)' (column 2)]
```



Patsy를 이용해서 모델 생성하기 > Patsy 용법으로 데이터 변환하기

Creating Model Descriptions with Patsy

Because the plus symbol (+) in the context of Patsy formulas does not mean addition, when you want to add columns from a dataset by name, you must wrap them in the special *I* function:

```
In [49]: y, X = patsy.dmatrices('y ~ I(x0 + x1)', data)
```

```
In [50]: X
```

```
Out[50]:
```

```
DesignMatrix with shape (5, 2)
```

```
Intercept  I(x0 + x1)
```

```
1          1.01
```

```
1          1.99
```

```
1          3.25
```

```
1         -0.10
```

```
1          5.00
```

```
Terms:
```

```
'Intercept' (column 0)
```

```
'I(x0 + x1)' (column 1)
```

Patsy has several other built-in transforms in the `patsy.builtins` module. See the [online documentation](#) for more.

Categorical data has a special class of transformations, which I explain next.



Patsy를 이용해서 모델 생성하기 > 범주형 데이터와 Patsy

Creating Model Descriptions with Patsy

Categorical Data and Patsy

Non-numeric data can be transformed for a model design matrix in many different ways. A complete treatment of this topic is outside the scope of this book and would be best studied along with a course in statistics.

When you use non-numeric terms in a Patsy formula, they are converted to dummy variables by default. If there is an intercept, one of the levels will be left out to avoid collinearity:

```
In [51]: data = pd.DataFrame({
.....:     'key1': ['a', 'a', 'b', 'b', 'a', 'b', 'a', 'b'],
.....:     'key2': [0, 1, 0, 1, 0, 1, 0, 0],
.....:     'v1': [1, 2, 3, 4, 5, 6, 7, 8],
.....:     'v2': [-1, 0, 2.5, -0.5, 4.0, -1.2, 0.2, -1.7]
.....: })
```

```
In [52]: y, X = patsy.dmatrices('v2 ~ key1', data)
```

```
In [53]: X
```

```
Out[53]:
```

```
DesignMatrix with shape (8, 2)
```

Intercept	key1[T.b]
1	0
1	0
1	1
1	1
1	0
1	1
1	0
1	1

```
Terms:
```

```
'Intercept' (column 0)
'key1' (column 1)
```

If you omit the intercept from the model, then columns for each category value will be included in the model design matrix:

```
In [54]: y, X = patsy.dmatrices('v2 ~ key1 + 0', data)
```

```
In [55]: X
```

```
Out[55]:
```

```
DesignMatrix with shape (8, 2)
```

key1[a]	key1[b]
1	0
1	0
0	1
0	1
1	0
0	1
1	0
0	1

```
Terms:
```

```
'key1' (columns 0:2)
```

Numeric columns can be interpreted as categorical with the C function:

```
In [56]: y, X = patsy.dmatrices('v2 ~ C(key2)', data)
```

```
In [57]: X
```

```
Out[57]:
```

```
DesignMatrix with shape (8, 2)
```

Intercept	C(key2)[T.1]
1	0
1	1
1	0
1	1
1	0
1	1
1	0
1	0

```
Terms:
```

```
'Intercept' (column 0)
'C(key2)' (column 1)
```



Patsy를 이용해서 모델 생성하기 > 범주형 데이터와 Patsy

Creating Model Descriptions with Patsy

When you're using multiple categorical terms in a model, things can be more complicated, as you can include interaction terms of the form `key1:key2`, which can be used, for example, in analysis of variance (ANOVA) models:

```
In [58]: data['key2'] = data['key2'].map({0: 'zero', 1: 'one'})
```

```
In [59]: data
```

```
Out[59]:
```

	key1	key2	v1	v2
0	a	zero	1	-1.0
1	a	one	2	0.0
2	b	zero	3	2.5
3	b	one	4	-0.5
4	a	zero	5	4.0
5	b	one	6	-1.2
6	a	zero	7	0.2
7	b	zero	8	-1.7

```
In [60]: y, X = patsy.dmatrices('v2 ~ key1 + key2', data)
```

```
In [61]: X
```

```
Out[61]:
```

```
DesignMatrix with shape (8, 3)
Intercept  key1[T.b]  key2[T.zero]
1          0          1
1          0          0
1          1          1
1          1          0
1          0          1
1          1          0
1          0          1
1          1          1
```

Terms:

```
'Intercept' (column 0)
'key1' (column 1)
'key2' (column 2)
```

```
In [62]: y, X = patsy.dmatrices('v2 ~ key1 + key2 + key1:key2', data)
```

```
In [63]: X
```

```
Out[63]:
```

```
DesignMatrix with shape (8, 4)
Intercept  key1[T.b]  key2[T.zero]  key1[T.b]:key2[T.zero]
1          0          1          0
1          0          0          0
1          1          1          1
1          1          0          0
1          0          1          0
1          1          0          0
1          0          1          0
1          1          1          1
```

Terms:

```
'Intercept' (column 0)
'key1' (column 1)
'key2' (column 2)
'key1:key2' (column 3)
```

Patsy provides for other ways to transform categorical data, including transformations for terms with a particular ordering. See the online documentation for more.



Patsy Documentation

<https://patsy.readthedocs.io/en/latest/API-reference.html#basic-api>



statsmodels 소개

Introduction to statsmodels

13.3 Introduction to statsmodels

statsmodels is a Python library for fitting many kinds of statistical models, performing statistical tests, and data exploration and visualization. Statsmodels contains more “classical” frequentist statistical methods, while Bayesian methods and machine learning models are found in other libraries.

Some kinds of models found in statsmodels include:

- Linear models, generalized linear models, and robust linear models
- Linear mixed effects models
- Analysis of variance (ANOVA) methods
- Time series processes and state space models
- Generalized method of moments

In the next few pages, we will use a few basic tools in statsmodels and explore how to use the modeling interfaces with Patsy formulas and pandas DataFrame objects.



statsmodels 소개 > 선형 모델 예측하기

Introduction to statsmodels

Estimating Linear Models

There are several kinds of linear regression models in statsmodels, from the more basic (e.g., ordinary least squares) to more complex (e.g., iteratively reweighted least squares).

Linear models in statsmodels have two different main interfaces: array-based and formula-based. These are accessed through these API module imports:

```
import statsmodels.api as sm
import statsmodels.formula.api as smf
```

To show how to use these, we generate a linear model from some random data:

```
def dnorm(mean, variance, size=1):
    if isinstance(size, int):
        size = size,
    return mean + np.sqrt(variance) * np.random.randn(*size)

# For reproducibility
np.random.seed(12345)

N = 100
X = np.c_[dnorm(0, 0.4, size=N),
          dnorm(0, 0.6, size=N),
          dnorm(0, 0.2, size=N)]
eps = dnorm(0, 0.1, size=N)
beta = [0.1, 0.3, 0.5]

y = np.dot(X, beta) + eps
```

Here, I wrote down the “true” model with known parameters beta. In this case, dnorm is a helper function for generating normally distributed data with a particular mean and variance. So now we have:

```
In [66]: X[:5]
Out[66]:
array([[ -0.1295, -1.2128,  0.5042],
       [  0.3029, -0.4357, -0.2542],
       [ -0.3285, -0.0253,  0.1384],
       [ -0.3515, -0.7196, -0.2582],
       [  1.2433, -0.3738, -0.5226]])
```

```
In [67]: y[:5]
Out[67]: array([ 0.4279, -0.6735, -0.0909, -0.4895, -0.1289])
```

A linear model is generally fitted with an intercept term as we saw before with Patsy. The sm.add_constant function can add an intercept column to an existing matrix:

```
In [68]: X_model = sm.add_constant(X)

In [69]: X_model[:5]
Out[69]:
array([[ 1. , -0.1295, -1.2128,  0.5042],
       [ 1. ,  0.3029, -0.4357, -0.2542],
       [ 1. , -0.3285, -0.0253,  0.1384],
       [ 1. , -0.3515, -0.7196, -0.2582],
       [ 1. ,  1.2433, -0.3738, -0.5226]])
```



statsmodels 소개 > 선형 모델 예측하기

Introduction to statsmodels

The `sm.OLS` class can fit an ordinary least squares linear regression:

```
In [70]: model = sm.OLS(y, X)
```

The model's `fit` method returns a regression results object containing estimated model parameters and other diagnostics:

```
In [71]: results = model.fit()
```

```
In [72]: results.params
Out[72]: array([ 0.1783,  0.223 ,  0.501 ])
```

The `summary` method on `results` can print a model detailing diagnostic output of the model:

```
In [73]: print(results.summary())
```

```
OLS Regression Results
=====
Dep. Variable:          y      R-squared:          0.430
Model:                  OLS      Adj. R-squared:      0.413
Method:                 Least Squares      F-statistic:      24.42
Date:                   Mon, 25 Sep 2017      Prob (F-statistic): 7.44e-12
Time:                   14:06:15      Log-Likelihood:    -34.305
No. Observations:       100      AIC:              74.61
Df Residuals:           97      BIC:              82.42
Df Model:                3
Covariance Type:        nonrobust
=====
                    coef    std err          t      P>|t|      [0.025    0.975]
-----
x1                0.1783     0.053      3.364     0.001     0.073     0.283
x2                0.2230     0.046      4.818     0.000     0.131     0.315
x3                0.5010     0.080      6.237     0.000     0.342     0.660
=====
Omnibus:                 4.662    Durbin-Watson:      2.201
Prob(Omnibus):           0.097    Jarque-Bera (JB):    4.098
Skew:                   0.481    Prob(JB):            0.129
Kurtosis:                3.243    Cond. No.            1.74
=====

Warnings:
[1] Standard Errors assume that the covariance matrix of the errors is correctly
specified.
```

The parameter names here have been given the generic names `x1`, `x2`, and so on. Suppose instead that all of the model parameters are in a `DataFrame`:

```
In [74]: data = pd.DataFrame(X, columns=['col0', 'col1', 'col2'])
```

```
In [75]: data['y'] = y
```

```
In [76]: data[:5]
Out[76]:
   col0    col1    col2     y
0 -0.129468 -1.212753  0.504225  0.427863
1  0.302910 -0.435742 -0.254180 -0.673480
2 -0.328522 -0.025302  0.138351 -0.090878
3 -0.351475 -0.719605 -0.258215 -0.489494
4  1.243269 -0.373799 -0.522629 -0.128941
```

Now we can use the `statsmodels` formula API and `Patsy` formula strings:

```
In [77]: results = smf.ols('y ~ col0 + col1 + col2', data=data).fit()
```

```
In [78]: results.params
```

```
Out[78]:
Intercept    0.033559
col0         0.176149
col1         0.224826
col2         0.514808
dtype: float64
```

```
In [79]: results.tvalues
```

```
Out[79]:
Intercept    0.952188
col0         3.319754
col1         4.850730
col2         6.303971
dtype: float64
```



statsmodels 소개 > 선형 모델 예측하기

Introduction to statsmodels

Observe how statsmodels has returned results as Series with the DataFrame column names attached. We also do not need to use `add_constant` when using formulas and pandas objects.

Given new out-of-sample data, you can compute predicted values given the estimated model parameters:

```
In [80]: results.predict(data[:5])
Out[80]:
0    -0.002327
1    -0.141904
2     0.041226
3    -0.323070
4    -0.100535
dtype: float64
```

There are many additional tools for analysis, diagnostics, and visualization of linear model results in statsmodels that you can explore. There are also other kinds of linear models beyond ordinary least squares.



statsmodels 소개 > 시계열 처리 예측

Introduction to statsmodels

Estimating Time Series Processes

Another class of models in statsmodels are for time series analysis. Among these are autoregressive processes, Kalman filtering and other state space models, and multi-variate autoregressive models.

Let's simulate some time series data with an autoregressive structure and noise:

```
init_x = 4

import random
values = [init_x, init_x]
N = 1000

b0 = 0.8
b1 = -0.4
noise = dnorm(0, 0.1, N)
for i in range(N):
    new_x = values[-1] * b0 + values[-2] * b1 + noise[i]
    values.append(new_x)
```

This data has an AR(2) structure (two *lags*) with parameters 0.8 and -0.4. When you fit an AR model, you may not know the number of lagged terms to include, so you can fit the model with some larger number of lags:

```
In [82]: MAXLAGS = 5
```

```
In [83]: model = sm.tsa.AR(values)
```

```
In [84]: results = model.fit(MAXLAGS)
```

The estimated parameters in the results have the intercept first and the estimates for the first two lags next:

```
In [85]: results.params
```

```
Out[85]: array([-0.0062,  0.7845, -0.4085, -0.0136,  0.015 ,  0.0143])
```

Deeper details of these models and how to interpret their results is beyond what I can cover in this book, but there's plenty more to discover in the statsmodels documentation.



statsmodels 소개 > 시계열 처리 예측

Introduction to statsmodels

Estimating Time Series Processes

Another class of models in statsmodels are for time series analysis. Among these are autoregressive processes, Kalman filtering and other state space models, and multi-variate autoregressive models.

Let's simulate some time series data with an autoregressive structure and noise:

```
init_x = 4

import random
values = [init_x, init_x]
N = 1000

b0 = 0.8
b1 = -0.4
noise = dnorm(0, 0.1, N)
for i in range(N):
    new_x = values[-1] * b0 + values[-2] * b1 + noise[i]
    values.append(new_x)
```

This data has an AR(2) structure (two *lags*) with parameters 0.8 and -0.4. When you fit an AR model, you may not know the number of lagged terms to include, so you can fit the model with some larger number of lags:

```
In [82]: MAXLAGS = 5
```

```
In [83]: model = sm.tsa.AR(values)
```

```
In [84]: results = model.fit(MAXLAGS)
```

The estimated parameters in the results have the intercept first and the estimates for the first two lags next:

```
In [85]: results.params
```

```
Out[85]: array([-0.0062,  0.7845, -0.4085, -0.0136,  0.015 ,  0.0143])
```

Deeper details of these models and how to interpret their results is beyond what I can cover in this book, but there's plenty more to discover in the statsmodels documentation.



statsmodels Documentation

<https://www.statsmodels.org/stable/api.html>



scikit-learn 소개

Introduction to scikit-learn

13.4 Introduction to scikit-learn

scikit-learn is one of the most widely used and trusted general-purpose Python machine learning toolkits. It contains a broad selection of standard supervised and unsupervised machine learning methods with tools for model selection and evaluation, data transformation, data loading, and model persistence. These models can be used for classification, clustering, prediction, and other common tasks.

There are excellent online and printed resources for learning about machine learning and how to apply libraries like scikit-learn and TensorFlow to solve real-world problems. In this section, I will give a brief flavor of the scikit-learn API style.

At the time of this writing, scikit-learn does not have deep pandas integration, though there are some add-on third-party packages that are still in development. pandas can be very useful for massaging datasets prior to model fitting, though.

As an example, I use a **now-classic dataset from a Kaggle competition** about passenger survival rates on the *Titanic*, which sank in 1912. We load the test and training dataset using pandas:

```
In [86]: train = pd.read_csv('datasets/titanic/train.csv')
```

```
In [87]: test = pd.read_csv('datasets/titanic/test.csv')
```

```
In [88]: train[:4]
```

```
Out[88]:
```

	PassengerId	Survived	Pclass	\
0	1	0	3	
1	2	1	1	
2	3	1	3	
3	4	1	1	

	Name	Sex	Age	SibSp	\
0	Braund, Mr. Owen Harris	male	22.0	1	
1	Cumings, Mrs. John Bradley (Florence Briggs Th...	female	38.0	1	
2	Heikkinen, Miss. Laina	female	26.0	0	
3	Futrelle, Mrs. Jacques Heath (Lily May Peel)	female	35.0	1	

	Parch	Ticket	Fare	Cabin	Embarked
0	0	A/5 21171	7.2500	NaN	S
1	0	PC 17599	71.2833	C85	C
2	0	STON/O2. 3101282	7.9250	NaN	S
3	0	113803	53.1000	C123	S



scikit-learn 소개

Introduction to scikit-learn

Libraries like statsmodels and scikit-learn generally cannot be fed missing data, so we look at the columns to see if there are any that contain missing data:

```
In [89]: train.isnull().sum()
Out[89]:
PassengerId      0
Survived          0
Pclass           0
Name             0
Sex              0
Age            177
SibSp            0
Parch           0
Ticket           0
Fare             0
Cabin          687
Embarked         2
dtype: int64
```

```
In [90]: test.isnull().sum()
Out[90]:
PassengerId      0
Pclass           0
Name             0
Sex              0
Age             86
SibSp            0
Parch           0
Ticket           0
Fare             1
Cabin          327
Embarked         0
dtype: int64
```

In statistics and machine learning examples like this one, a typical task is to predict whether a passenger would survive based on features in the data. A model is fitted on a *training* dataset and then evaluated on an out-of-sample *testing* dataset.

I would like to use Age as a predictor, but it has missing data. There are a number of ways to do missing data *imputation*, but I will do a simple one and use the median of the training dataset to fill the nulls in both tables:

```
In [91]: impute_value = train['Age'].median()

In [92]: train['Age'] = train['Age'].fillna(impute_value)

In [93]: test['Age'] = test['Age'].fillna(impute_value)
```

Now we need to specify our models. I add a column `IsFemale` as an encoded version of the 'Sex' column:

```
In [94]: train['IsFemale'] = (train['Sex'] == 'female').astype(int)

In [95]: test['IsFemale'] = (test['Sex'] == 'female').astype(int)
```

Then we decide on some model variables and create NumPy arrays:

```
In [96]: predictors = ['Pclass', 'IsFemale', 'Age']

In [97]: X_train = train[predictors].values

In [98]: X_test = test[predictors].values

In [99]: y_train = train['Survived'].values

In [100]: X_train[:5]
Out[100]:
array([[ 3.,  0., 22.],
       [ 1.,  1., 38.],
       [ 3.,  1., 26.],
       [ 1.,  1., 35.],
       [ 3.,  0., 35.]])

In [101]: y_train[:5]
Out[101]: array([0, 1, 1, 1, 0])
```



scikit-learn 소개

Introduction to scikit-learn

I make no claims that this is a good model nor that these features are engineered properly. We use the LogisticRegression model from scikit-learn and create a model instance:

```
In [102]: from sklearn.linear_model import LogisticRegression
```

```
In [103]: model = LogisticRegression()
```

Similar to statsmodels, we can fit this model to the training data using the model's fit method:

```
In [104]: model.fit(X_train, y_train)
```

```
Out[104]:
```

```
LogisticRegression(C=1.0, class_weight=None, dual=False, fit_intercept=True,
    intercept_scaling=1, max_iter=100, multi_class='ovr', n_jobs=1,
    penalty='l2', random_state=None, solver='liblinear', tol=0.0001,
    verbose=0, warm_start=False)
```

Now, we can form predictions for the test dataset using model.predict:

```
In [105]: y_predict = model.predict(X_test)
```

```
In [106]: y_predict[:10]
```

```
Out[106]: array([0, 0, 0, 0, 1, 0, 1, 0, 1, 0])
```

If you had the true values for the test dataset, you could compute an accuracy percentage or some other error metric:

```
(y_true == y_predict).mean()
```

In practice, there are often many additional layers of complexity in model training. Many models have parameters that can be tuned, and there are techniques such as *cross-validation* that can be used for parameter tuning to avoid overfitting to the training data. This can often yield better predictive performance or robustness on new data.

Cross-validation works by splitting the training data to simulate out-of-sample prediction. Based on a model accuracy score like mean squared error, one can perform a grid search on model parameters. Some models, like logistic regression, have estimator classes with built-in cross-validation. For example, the LogisticRegressionCV class can be used with a parameter indicating how fine-grained of a grid search to do on the model regularization parameter C:

```
In [107]: from sklearn.linear_model import LogisticRegressionCV
```

```
In [108]: model_cv = LogisticRegressionCV(10)
```

```
In [109]: model_cv.fit(X_train, y_train)
```

```
Out[109]:
```

```
LogisticRegressionCV(Cs=10, class_weight=None, cv=None, dual=False,
    fit_intercept=True, intercept_scaling=1.0, max_iter=100,
    multi_class='ovr', n_jobs=1, penalty='l2', random_state=None,
    refit=True, scoring=None, solver='lbfgs', tol=0.0001, verbose=0)
```



scikit-learn 소개

Introduction to scikit-learn

To do cross-validation by hand, you can use the `cross_val_score` helper function, which handles the data splitting process. For example, to cross-validate our model with four non-overlapping splits of the training data, we can do:

```
In [110]: from sklearn.model_selection import cross_val_score

In [111]: model = LogisticRegression(C=10)

In [112]: scores = cross_val_score(model, X_train, y_train, cv=4)

In [113]: scores
Out[113]: array([ 0.7723,  0.8027,  0.7703,  0.7883])
```

The default scoring metric is model-dependent, but it is possible to choose an explicit scoring function. Cross-validated models take longer to train, but can often yield better model performance.



scikit-learn Documentation

<https://scikit-learn.org/stable/modules/classes.html>



더 공부하기

Continuing Your Education

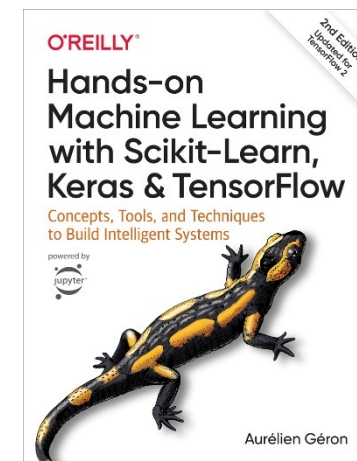
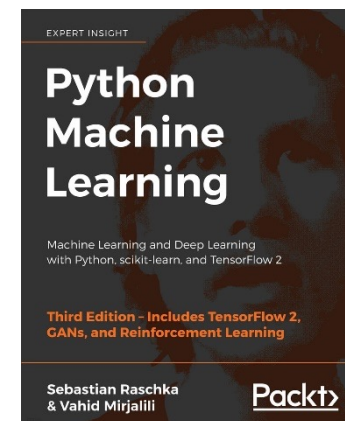
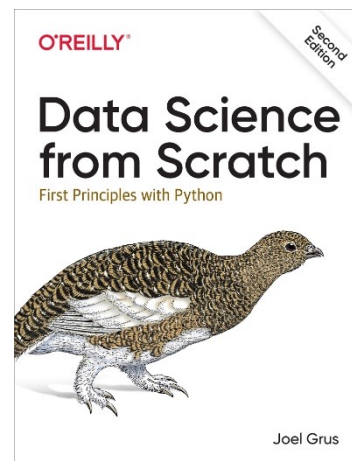
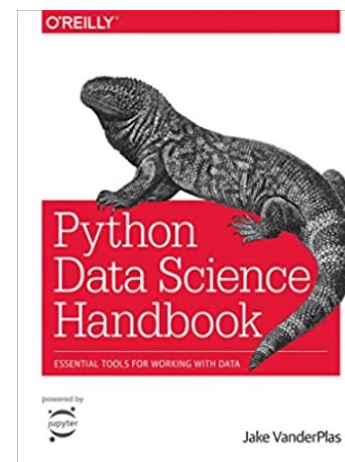
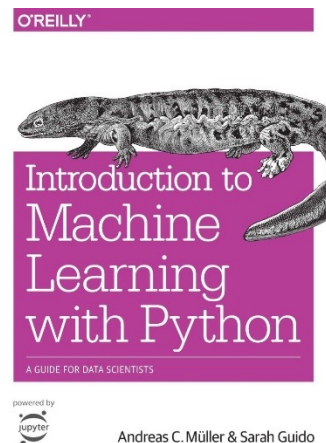
13.5 Continuing Your Education

While I have only skimmed the surface of some Python modeling libraries, there are more and more frameworks for various kinds of statistics and machine learning either implemented in Python or with a Python user interface.

This book is focused especially on data wrangling, but there are many others dedicated to modeling and data science tools. Some excellent ones are:

- *Introduction to Machine Learning with Python* by Andreas Mueller and Sarah Guido (O'Reilly)
- *Python Data Science Handbook* by Jake VanderPlas (O'Reilly)
- *Data Science from Scratch: First Principles with Python* by Joel Grus (O'Reilly)
- *Python Machine Learning* by Sebastian Raschka (Packt Publishing)
- *Hands-On Machine Learning with Scikit-Learn and TensorFlow* by Aurélien Géron (O'Reilly)

While books can be valuable resources for learning, they can sometimes grow out of date when the underlying open source software changes. It's a good idea to be familiar with the documentation for the various statistics or machine learning frameworks to stay up to date on the latest features and API.



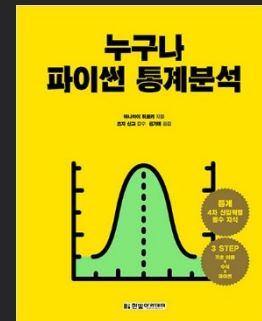
Q&A and Break Time

질의응답 및 휴식시간 (10분)



2부: 독립성 & 점/구간 추정

- 1) 독립성
- 2) 점/구간 추정





독립성

1 독립성 정의 (독립성 = 무상관성 ?)



독립성

1. 독립성 정의 (독립성 = 무상관성 ?)

이전까지 다루었던 확률변수 : 1차원, 2차원 확률변수

이번 수업부터 다루게 될 확률변수 : 서로 독립인 다차원 확률변수

- 독립이란?
 - 확률변수가 다른 확률변수에 서로 영향을 미치지 않는 상태
- 독립이란 개념의 중요성
 - 통계분석에서 사용하는 데이터의 대부분이 독립이면서 다차원인 확률변수로 가정하고 분석을 진행하므로



독립성

1. 독립성 정의 (독립성 = 무상관성 ?)

.

서로 독립인 다차원 확률변수

- 예시

- 400명의 전교생의 평균 점수를 파악하기 위해,
우연히 만난 임의의 학생 20명에게 시험점수를 물어보고 평균 점수를 추측했다고 하자
- 우연히 만난 학생 한 개인의 입장에서의 점수 각각은 전교생의 점수 분포(X_n)를 따르는 1차원 확률변수이다
- 또한 우연히 만남으로서 학생들 각각은 서로간 관계가 없는 상태(독립)로 추출이 되었으므로,
최종적으로 $(X_1, X_2, \dots, X_{20})$ 은 서로 독립인 20차원의 확률변수라 표현이 가능하다

- 독립동일분포란?

- 위 예시처럼 $(X_1, X_2, \dots, X_{20})$ 은 서로 독립이고 각각 동일한 확률분포를 따르는 다차원 확률변수를 독립동일분포(i.i.d : Independently and identically distributed)라 한다
- 독립동일분포는 통계분석에서 가장 기본적이고 중요한 조건이므로 꼭 알아두길 권장한다



독립성

1. 독립성 정의 (독립성 = 무상관성 ?)

.

파이썬 기초 라이브러리 세팅

- Numpy (분포 구현)
- Matplotlib의 pyplot 모듈 (시각화)
- Scipy의 stats 모듈 (통계)

```
1  import numpy as np
2  import matplotlib.pyplot as plt
3  from scipy import stats
4
5  np.random.seed(0)
6  %precision 3
7  %matplotlib inline
```



독립성

1. 독립성 정의 (독립성 = 무상관성 ?)

.

통계 계산 함수 활용

- $E(XY)$: 2차원 확률변수의 기대값을 계산하는 함수
- $Cov(XY)$: 2차원 확률변수의 공분산을 계산하는 함수
- $f_X(x), f_Y(y)$: X와 Y의 주변확률밀도함수

```
1  linestyles = ['-', '--', ':', '-.']
2
3  √def E(XY, g):
4      x_set, y_set, f_XY = XY
5      √return np.sum([g(x_i, y_j) * f_XY(x_i, y_j)
6                      for x_i in x_set for y_j in y_set])
7
8  √def Cov(XY):
9      x_set, y_set, f_XY = XY
10     mean_X = E(XY, lambda x, y: x)
11     mean_Y = E(XY, lambda x, y: y)
12     √return np.sum([(x_i-mean_X) * (y_j-mean_Y) * f_XY(x_i, y_j)
13                     for x_i in x_set for y_j in y_set])
14
15  √def f_X(x):
16      return np.sum([f_XY(x, y_k) for y_k in y_set])
17
18  √def f_Y(y):
19      return np.sum([f_XY(x_k, y) for x_k in x_set])
```



독립성

1. 독립성 정의 (독립성 = 무상관성 ?)

.

독립성의 정의

- 확률변수의 '독립성(Independence)'이란,
2개 이상의 확률변수가 서로 영향을 끼치지 않으면서 관계가 없음을 나타내는 개념
- 2차원 확률변수 (X, Y) 의 경우, 다음과 같은 관계식이 성립할 때가 독립!

$$f_{X,Y}(x, y) = f_X(x)f_Y(y)$$

(즉, 확률변수가 독립이면 결합확률은 주변확률밀도함수의 곱으로 표현된다)



독립성

1. 독립성 정의 (독립성 = 무상관성 ?)

.

독립성 예시

- 불공정한 주사위 A와 B에서 나온 눈을 각각 확률변수 X와 Y라고 표현할 때, A의 눈과 B의 눈은 서로 영향을 주지 않으며 또한 관계가 없는 독립이다
- (X,Y)의 결합확률과 X와 Y의 주변확률함수를 수식으로 표현해보면 아래와 같다

$$f_{X,Y}(x,y) = \begin{cases} \frac{xy}{441} & (x \in \{1,2,3,4,5,6\} \text{ 및 } y \in \{1,2,3,4,5,6\}) \\ 0 & (otherwise) \end{cases}$$

$$f_X(x) = \begin{cases} \frac{x}{21} & (x \in \{1,2,3,4,5,6\}) \\ 0 & (otherwise) \end{cases}, \quad f_Y(y) = \begin{cases} \frac{y}{21} & (y \in \{1,2,3,4,5,6\}) \\ 0 & (otherwise) \end{cases}$$

(즉, $f_{X,Y}(x,y) = f_X(x)f_Y(y)$ 가 성립한다는 것을 확인할 수 있다)



독립성

1. 독립성 정의 (독립성 = 무상관성 ?)

.

독립성의 일반화

- n 개의 확률변수 $(X_1, X_2, \dots, X_n)$ 이 다음을 만족할 때, $X_1, X_2, \dots, X_n$ 은 서로 독립이다

$$f_{X_1, X_2, \dots, X_n}(x_1, x_2, \dots, x_n) = f_{X_1}(x_1)f_{X_2}(x_2) \cdots f_{X_n}(x_n)$$

다만 f 가 이산형이면 주변확률함수, 연속형이면 주변확률밀도함수를 나타낸다



독립성

1. 독립성 정의 (독립성 = 무상관성 ?)

.

독립성과 무상관성

- Ch5와 Ch7에서 배웠던 공분산과 상관계수의 개념은
그 값이 0일 때 무상관이라 표현한다
즉, 선형 관계(상관성)가 없음을 나타낸다
- 독립성과 무상관성의 차이

독립성 $\in$ 무상관성

즉, 독립일 때는 무상관성을 만족하지만, 무상관이라고 해서 반드시 독립인 것은 아니다



독립성

1. 독립성 정의 (독립성 = 무상관성 ?)

.

[Python] 독립성과 무상관성 예시 확인

- 불공정한 주사위 A, B의 눈이 각각 X, Y 확률변수로 표현될 때
 - 공분산이 0으로서, 독립인 예시임에도 무상관을 만족함을 알 수 있다.

```
1  x_set = np.array([1, 2, 3, 4, 5, 6])
2  y_set = np.array([1, 2, 3, 4, 5, 6])
3
4  def f_XY(x, y):
5      if x in x_set and y in y_set:
6          return x * y / 441
7      else:
8          return 0
9
10 XY = [x_set, y_set, f_XY]
```

```
1  Cov(XY)
```

```
-0.000
```



독립성

1. 독립성 정의 (독립성 = 무상관성 ?)

[Python] 독립성과 무상관성 예시 확인

- $\{(0,0), (1,1), (1,-1)\}$ 로 각각 확률이 동일한 2차원 확률변수 (X,Y) 가 있다고 할 때

$$f_{X,Y}(x,y) = \begin{cases} \frac{1}{3} & (x,y) \in \{(0,0), (1,1), (1,-1)\} \\ 0 & (otherwise) \end{cases}$$

- 공분산 계산시 무상관성 확인

```
1 x_set = np.array([0, 1])
2 y_set = np.array([-1, 0, 1])
3
4 def f_XY(x, y):
5     if (x, y) in [(0, 0), (1, 1), (1, -1)]:
6         return 1 / 3
7     else:
8         return 0
9
10 XY = [x_set, y_set, f_XY]
```

```
1 Cov(XY)
0.000
```

- 독립성의 정의 계산시 독립이 아님을 확인

```
1 f_X(0) * f_Y(0), f_XY(0, 0)
(0.111, 0.333)
```

- 즉, 무상관성을 만족함에도 독립은 아닐 수 있다



독립성

2 합의 분포



합의 분포 파악이 필요한 이유

- 400명의 전교생 중 우연히 만난 20명의 점수 분포 $(X_1, X_2, \dots, X_{20})$ 를 기반으로 모평균을 추정하기 위해 사용되는 표본평균은 아래 식을 따른다

$$\bar{X} = \frac{X_1 + X_2 + \dots + X_{20}}{20}$$

- 표본 평균에 대한 이해도를 높이기 위한 사전 준비를 하는 차원에서 아래와 같은 형태의 합의 분포에 대한 파악이 먼저 필요하다

$$\sum_{i=1}^n X_i = X_1 + X_2 + \dots + X_n$$

- 그 후, n 으로 나눈 표본평균에 대해서 쉽게 이해할 수 있다



합의 분포

$$\sum_{i=1}^n X_i = X_1 + X_2 + \cdots + X_n$$

- 합의 분포의 이산형 확률함수, 연속형 확률밀도함수를 직접 유도하는 것은 어렵다
- 따라서, 합의 분포의 기대값과 분산에 대해 먼저 파악해보도록한다



독립성

2. 합의 분포

합의 분포의 기대값

- 확률변수 $X_1, X_2, \dots, X_n$ 에 관하여 아래 식이 성립한다

$$E(X_1 + X_2 + \dots + X_n) = E(X_1) + E(X_2) + \dots + E(X_n)$$

- Chapter 5에서 다룬 것처럼 선형성이 성립하기 때문

*** 확률변수가 서로 독립이 아니어도 성립**

합의 분포의 분산

- 확률변수 $X_1, X_2, \dots, X_n$ 이 **서로 독립**이라면, 아래 식이 성립한다

$$V(X_1 + X_2 + \dots + X_n) = V(X_1) + V(X_2) + \dots + V(X_n)$$

- Chapter 5에서 다룬 것처럼 확률변수간 공분산이 없었으므로

독립성이 만족될 때, 상관성이 0(공분산이 0)이 되어 선형성이 성립하게 된다



합의 분포를 기대값과 분산이 아닌 실제 확률 분포의 형태를 알고 싶다면?

- 대부분의 경우, 합의 분포의 구체적인 형태 정의가 어렵다
- 몇 가지 대표 확률분포들은 합의 분포($\sum_{i=1}^n X_i$) 또한 대표적인 형태의 확률분포를 따르는 것으로 알려져있다
 - 1) 정규분포의 합의 분포 -> 정규분포
 - 2) 푸아송 분포의 합의 분포 -> 푸아송 분포
 - 3) 베르누이 분포의 합의 분포 -> 이항분포



독립성

2. 합의 분포

정규분포의 합의 분포 예시

- 서로 독립인 확률변수 X, Y 는 각각 $X \sim N(1,2)$ 와 $Y \sim N(2,3)$ 로 정의되어있을 때
 - 1) $E(X+Y) = E(X) + E(Y) = 3$
 - 2) $V(X+Y) = V(X) + V(Y) = 5$를 만족한다

[Python] 100만개 샘플을 무작위 추출하여 기대값 및 분산의 유사성 확인

```
1  rv1 = stats.norm(1, np.sqrt(2))
2  rv2 = stats.norm(2, np.sqrt(3))
3
4  sample_size = int(1e6)
5  X_sample = rv1.rvs(sample_size)
6  Y_sample = rv2.rvs(sample_size)
7
8  sum_sample = X_sample + Y_sample
9
10 np.mean(sum_sample), np.var(sum_sample)

(3.003, 4.996)
```



☒☒☒

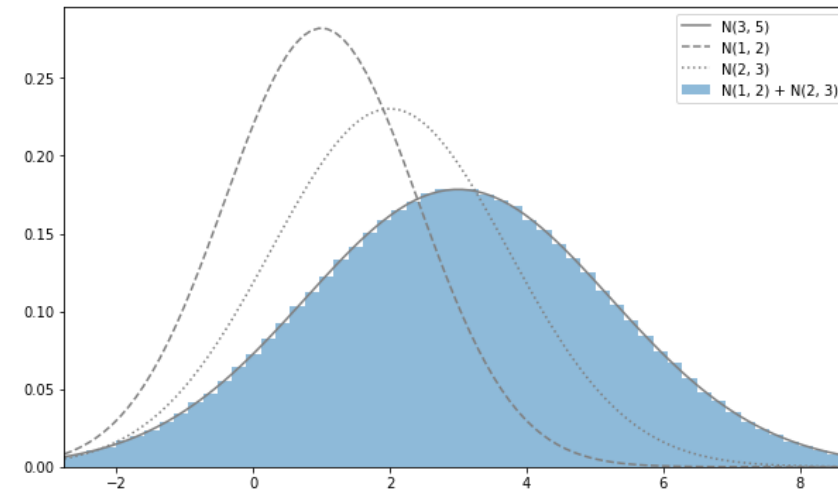
2. 합의 분포

☒☒☒☒☒ ☒☒ ☒☒

- 정규분포는 합의 분포 또한 정규분포가 되는 성질 보유
(재생성: 동일한 분포를 따르는 2개의 독립인 확률변수의 합 또한 동일한 확률분포가 되는 성질)

[Python] 정규분포의 합의 분포 시각화 $N(3, 5) = N(1, 2) + N(2, 3)$

```
1 fig = plt.figure(figsize=(10, 6))
2 ax = fig.add_subplot(111)
3
4 rv = stats.norm(3, np.sqrt(5))
5 xs = np.linspace(rv.isf(0.995), rv.isf(0.005), 100)
6
7 ax.hist(sum_sample, bins=100, density=True,
8         alpha=0.5, label='N(1, 2) + N(2, 3)')
9 ax.plot(xs, rv.pdf(xs), label='N(3, 5)', color='gray')
10 ax.plot(xs, rv1.pdf(xs), label='N(1, 2)', ls='--', color='gray')
11 ax.plot(xs, rv2.pdf(xs), label='N(2, 3)', ls=':', color='gray')
12
13 ax.legend()
14 ax.set_xlim(rv.isf(0.995), rv.isf(0.005))
15 plt.show()
```





☒☒☒

2. 합의 분포

☒☒☒☒☒ ☒☒ ☒☒ ☒☒☒

- 서로 독립인 확률변수 $X_1 \sim N(\mu_1, \sigma_1^2), X_2 \sim N(\mu_2, \sigma_2^2), \dots, X_n \sim N(\mu_n, \sigma_n^2)$ 에 관하여

$$\sum_{i=1}^n X_i \sim N\left(\sum_{i=1}^n \mu_i, \sum_{i=1}^n \sigma_i^2\right)$$

위 식이 성립한다



독립성

2. 합의 분포

푸아송 분포의 합의 분포 예시

- 서로 독립인 확률변수 $X \sim Poi(3)$ 과 $Y \sim Poi(4)$ 로 정의되어있을 때

1) $E(X+Y) = E(X) + E(Y) = 3+4 = 7$

2) $V(X+Y) = V(X) + V(Y) = 3+4 = 7$

을 만족한다

[Python] 100만개 샘플을 무작위 추출하여 기대값 및 분산의 유사성 확인

```
1 rv1 = stats.poisson(3)
2 rv2 = stats.poisson(4)
3
4 sample_size = int(1e6)
5 X_sample = rv1.rvs(sample_size)
6 Y_sample = rv2.rvs(sample_size)
7 sum_sample = X_sample + Y_sample
8
9 np.mean(sum_sample), np.var(sum_sample)

(6.999, 6.990)
```

푸아송 분포의 합의 분포

- 푸아송분포 또한 재생성을 기반으로 합의 분포가 푸아송분포가 됨

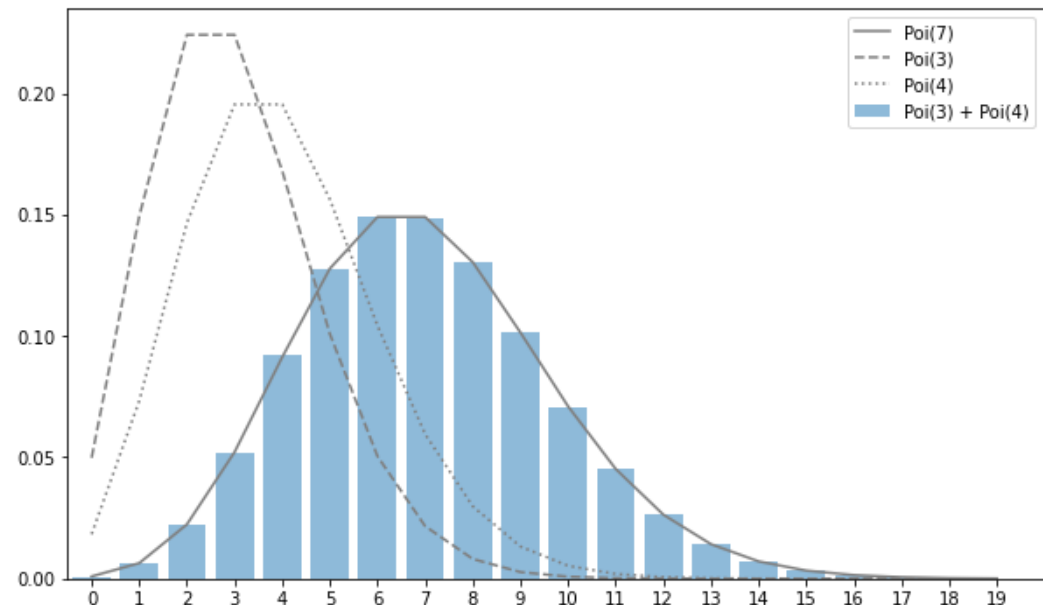
$$X + Y \sim \text{Poi}(7)$$

[Python] 푸아송 분포의 합의 분포 시각화 $\text{Poi}(7) = \text{Poi}(3) + \text{Poi}(4)$

```

1  fig = plt.figure(figsize=(10, 6))
2  ax = fig.add_subplot(111)
3
4  rv = stats.poisson(7)
5  xs = np.arange(20)
6  hist, _ = np.histogram(sum_sample, bins=20,
7                          range=(0, 20), normed=True)
8
9  ax.bar(xs, hist, alpha=0.5, label='Poi(3) + Poi(4)')
10 ax.plot(xs, rv.pmf(xs), label='Poi(7)', color='gray')
11 ax.plot(xs, rv1.pmf(xs), label='Poi(3)', ls='--', color='gray')
12 ax.plot(xs, rv2.pmf(xs), label='Poi(4)', ls=':', color='gray')
13
14 ax.legend()
15 ax.set_xlim(-0.5, 20)
16 ax.set_xticks(np.arange(20))
17 plt.show()

```





독립성

2. 합의 분포

.

푸아송 분포의 합의 분포 일반화

- 서로 독립인 확률변수 $X_1 \sim Poi(\lambda_1), X_2 \sim Poi(\lambda_2), \dots, X_n \sim Poi(\lambda_n)$ 에 관하여

$$\sum_{i=1}^n X_i \sim Poi\left(\sum_{i=1}^n \lambda_i\right)$$

위 식이 성립한다



베르누이 분포의 합의 분포 예시

- 서로 독립인 확률변수 $X_1, X_2, \dots, X_{10}$ 가 동일 분포 $Bern(0.3)$ 로 정의되어있을 때

$$1) E\left(\sum_{i=1}^{10} X_i\right) = E(X_1) + E(X_2) + \dots + E(X_{10}) = 10 \times 0.3 = 3$$

$$2) V\left(\sum_{i=1}^{10} X_i\right) = V(X_1) + V(X_2) + \dots + V(X_{10}) = 10 \times 0.3 \times (1 - 0.3) = 2.1$$

을 만족한다

[Python] 100만개 샘플을 무작위 추출하여 기대값 및 분산의 유사성 확인

```
1 p = 0.3
2 rv = stats.bernoulli(p)
3
4 sample_size = int(1e6)
5 Xs_sample = rv.rvs((10, sample_size))
6 sum_sample = np.sum(Xs_sample, axis=0)
7
8 np.mean(sum_sample), np.var(sum_sample)

(2.999, 2.095)
```

베르누이 분포의 합의 분포

- 베르누이 분포는 재생성이 없고, 합의 분포가 이항분포가 되는 성질이 있음

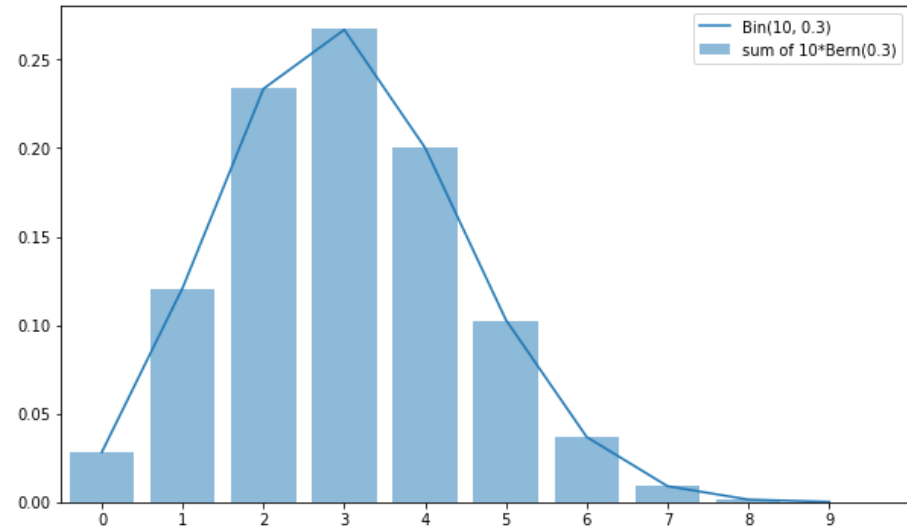
$$\sum_{i=1}^{10} X_i \sim \text{Bin}(10, 0.3)$$

[Python] 베르누이 분포의 합의 분포 시각화 $\text{Bin}(10, 0.3) = \sum_{i=1}^{10} \text{Bern}(0.3)$

```

1  fig = plt.figure(figsize=(10, 6))
2  ax = fig.add_subplot(111)
3
4  rv = stats.binom(10, p)
5  xs = np.arange(10)
6  hist, _ = np.histogram(sum_sample, bins=10,
7  range=(0, 10), normed=True)
8  ax.bar(xs, hist, alpha=0.5, label='sum of 10*Bern(0.3)')
9  ax.plot(xs, rv.pmf(xs), label='Bin(10, 0.3)')
10 ax.legend()
11 ax.set_xlim(-0.5, 10)
12 ax.set_xticks(np.arange(10))
13 plt.show()

```





□□□

2. □□ □□

.

□□□ □□□ □□ □□ □□□

- 서로 독립이면서 동일한 분포를 가지는 확률변수 $X_1, X_2, \dots, X_n \sim \text{Bern}(p)$ 에 관하여

$$\sum_{i=1}^n X_i \sim \text{Bin}(n, p)$$

위 식이 성립한다



독립성

3 표본평균의 분포



표본평균의 분포

- 서로 독립이고 동일한 확률분포를 따르는 확률변수 $X_1, X_2, \dots, X_n$ 의 표본평균

$$\bar{X} = \frac{X_1 + X_2 + \dots + X_n}{n}$$

이 따르는 분포

- Chapter 10 모평균의 구간추정, Chapter 11 모평균의 검정시 활용될 분포
- 중심극한정리 / 대수의 법칙 등의 성질을 포함하는 중요한 개념



독립성

3. 표본평균의 분포

표본평균 분포의 기대값과 분산 정의 및 계산

- 확률변수 $X_1, X_2, \dots, X_n$ 이 서로 독립이고 동일한 분포를 따르며, 기대값이 μ , 분산이 σ^2 일 때

$$E(\bar{X}) = \mu$$

$$V(\bar{X}) = \frac{\sigma^2}{n}$$

위 식이 성립한다

- 기대값 및 분산 계산

$$E(\bar{X}) = E\left(\frac{X_1 + X_2 + \dots + X_n}{n}\right) = \frac{E(X_1) + E(X_2) + \dots + E(X_n)}{n} = \frac{n\mu}{n} = \mu$$

$$V(\bar{X}) = V\left(\frac{X_1 + X_2 + \dots + X_n}{n}\right) = \frac{V(X_1) + V(X_2) + \dots + V(X_n)}{n^2} = \frac{n\sigma^2}{n^2} = \frac{\sigma^2}{n}$$



정규분포의 표본평균 분포 예시

- 10개의 확률변수가 서로 독립이면서, 동일한 분포 $X_1, X_2, \dots, X_{10} \sim N(1, 2)$ 를 만족할 때

$$E(\bar{X}) = 1, V(\bar{X}) = \frac{2}{10} \text{이 된다}$$

[Python] 100만개 샘플을 무작위 추출하여 기대값 및 분산의 유사성 확인

```
1 mean = 1
2 var = 2
3 rv = stats.norm(mean, np.sqrt(var))
4
5 n = 10
6 sample_size = int(1e6)
7 Xs_sample = rv.rvs((n, sample_size))
8 sample_mean = np.mean(Xs_sample, axis=0)
9
10 np.mean(sample_mean), np.var(sample_mean)

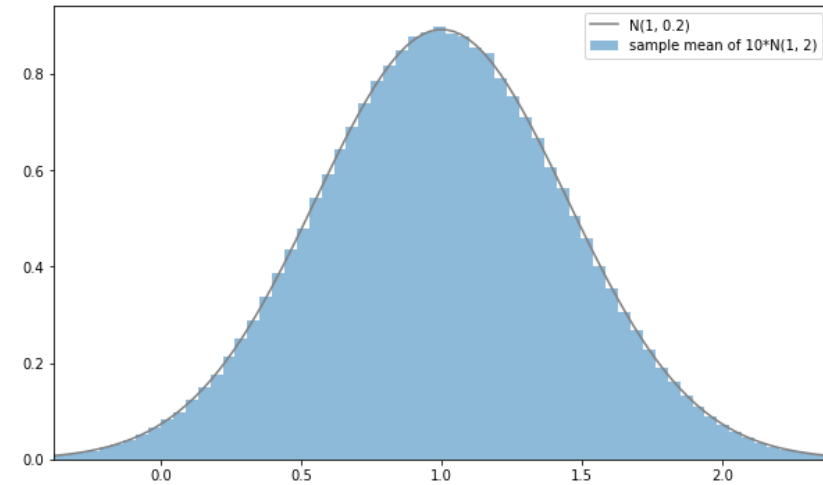
(1.000, 0.199)
```


정규분포의 표본평균 분포

- 정규분포는 합의 분포 또한 정규분포가 되므로 $\bar{X} \sim N(1, \frac{2}{10})$ 를 만족한다

[Python] 정규분포의 합의 분포 시각화 $\bar{X} \sim N(1, \frac{2}{10})$

```
1  fig = plt.figure(figsize=(10, 6))
2  ax = fig.add_subplot(111)
3
4  rv_true = stats.norm(mean, np.sqrt(var/n))
5  xs = np.linspace(rv_true.isf(0.999), rv_true.isf(0.001), 100)
6  ax.hist(sample_mean, bins=100, density=True,
7         | alpha=0.5, label='sample mean of 10*N(1, 2)')
8  ax.plot(xs, rv_true.pdf(xs), label='N(1, 0.2)', color='gray')
9
10 ax.legend()
11 ax.set_xlim(rv_true.isf(0.999), rv_true.isf(0.001))
12 plt.show()
```





독립성

3. 표본평균의 분포

.

정규분포의 표본평균 분포의 일반화

- n 개의 확률변수가 서로 독립이면서, 동일한 분포 $X_1, X_2, \dots, X_n \sim N(\mu, \sigma^2)$ 를 만족할 때

$$\bar{X} \sim N\left(\mu, \frac{\sigma^2}{n}\right)$$

위 식이 성립한다



푸아송 분포의 표본평균 분포 예시

- 10개의 확률변수가 서로 독립이면서, 동일한 분포 $X_1, X_2, \dots, X_{10} \sim Poi(3)$ 를 만족할 때

$$E(\bar{X}) = 3, V(\bar{X}) = \frac{3}{10} \text{이 된다}$$

[Python] 100만개 샘플을 무작위 추출하여 기대값 및 분산의 유사성 확인

```
1  l = 3
2  rv = stats.poisson(l)
3
4  n = 10
5  sample_size = int(1e6)
6  Xs_sample = rv.rvs((n, sample_size))
7  sample_mean = np.mean(Xs_sample, axis=0)
8
9  np.mean(sample_mean), np.var(sample_mean)

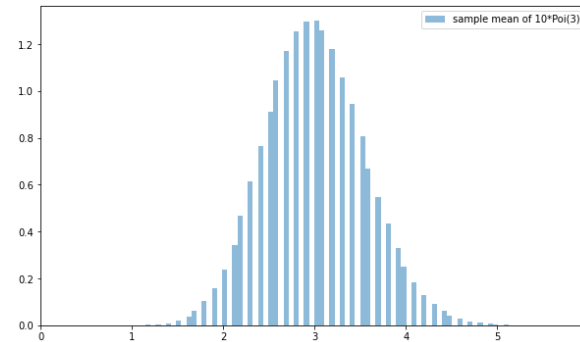
(2.999, 0.300)
```

푸아송 분포의 표본평균 분포

- 푸아송 분포는 합의 분포가 푸아송 분포가 되지만,
표본평균 $\bar{X}$ 는 기대값 3, 분산 0.3으로서 푸아송분포를 따르지 않는다

[Python] 푸아송 분포의 표본평균 분포 시각화 $\bar{X}$

```
1 fig = plt.figure(figsize=(10, 6))
2 ax = fig.add_subplot(111)
3
4 ax.hist(sample_mean, bins=100, density=True,
5         alpha=0.5, label='sample mean of 10*Poi(3)')
6
7 ax.legend()
8 ax.set_xlim(0, 6)
9 plt.show()
```



- 좌우대칭인 종 모형의 분포로 정규분포에 가까운 형태가 되는 것을 확인
 - 실제로 푸아송 분포의 표본분포는 근사적으로 정규분포를 따르는 것을 알 수 있다
 - 다음으로 배울 중심극한정리를 통해서 설명을 진행할 예정



독립성

3. 표본평균의 분포

.

합의 분포, 표본평균의 분포의 형태 파악의 어려움

- 합의 분포의 형태
 - 정규 분포 혹은 푸아송 분포의 경우까지는 재생성이 있어,
쉽게 합의 분포의 형태를 파악할 수 있음
- 표본평균 분포의 형태
 - 정규 분포를 제외하고는 푸아송 분포 조차도 그 형태가 유지되지 않음

=> 표본평균 분포의 형태를 파악하는 강력한 정리인 중심극한정리를 배울 필요가 있음



중심극한정리

- 확률변수 $X_1, X_2, \dots, X_n$ 이 서로 독립이고, 기대값이 μ , 분산이 σ^2 인 확률분포를 따를 때,
n이 커짐에 따라 표본평균 $\bar{X}$ 의 분포는 정규분포 $N\left(\mu, \frac{\sigma^2}{n}\right)$ 에 가까워진다

중심극한정리 예시

- 확률변수 $n=10000$ 개가 서로 독립이며 동일한 분포 $X_1, X_2, \dots, X_{10,000} \sim Poi(3)$ 를 따를 때,
중심극한정리에 의해 표본평균 $\bar{X}$ 의 분포는 근사적으로 정규분포 $N\left(3, \frac{3}{10,000}\right)$ 로 수렴한다

[Python] 중심극한정리 예시($X_1, X_2, \dots, X_{10,000} \sim \text{Poi}(3)$)의 표본평균 분포 시각화

- 10,000개 확률변수 별 10,000개씩 샘플 추출

```

1  l = 3
2  rv = stats.poisson(l)
3
4  n = 10000
5  sample_size = 10000
6  Xs_sample = rv.rvs((n, sample_size))
7  sample_mean = np.mean(Xs_sample, axis=0)
8
9  rv_true = stats.norm(l, np.sqrt(l/n))
10 xs = np.linspace(rv_true.isf(0.999), rv_true.isf(0.001), 100)

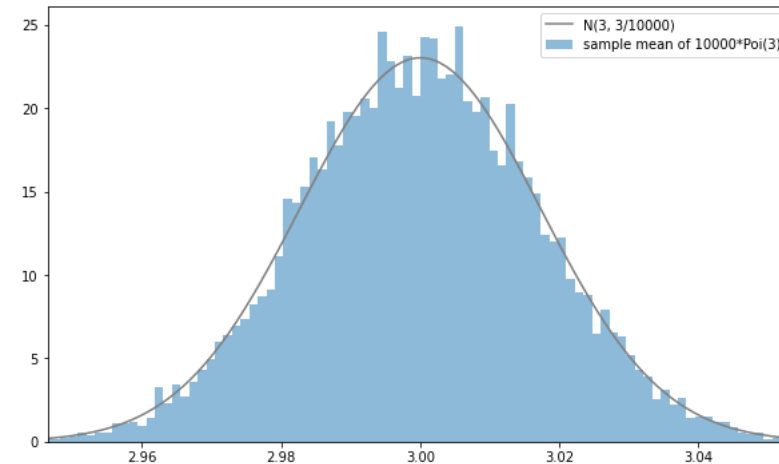
```

- $\bar{X} \sim N\left(3, \frac{3}{10,000}\right)$ 시각화

```

1  fig = plt.figure(figsize=(10, 6))
2  ax = fig.add_subplot(111)
3
4  ax.hist(sample_mean, bins=100, density=True,
5         alpha=0.5, label='sample mean of 10000*Poi(3)')
6  ax.plot(xs, rv_true.pdf(xs), label='N(3, 3/10000)', color='gray')
7
8  ax.legend()
9  ax.set_xlim(rv_true.isf(0.999), rv_true.isf(0.001))
10 plt.show()

```





대수의 법칙

- 확률변수 $X_1, X_2, \dots, X_n$ 이 서로 독립이고,
기대값이 μ , 분산이 σ^2 인 확률분포를 따를 때,
 n 이 커짐에 따라 표본평균은 μ 에 수렴한다
 - 즉, '표본 크기를 키우면 표본평균(추정)은 모평균(정답)에 수렴한다'라는 정리
 - 표본평균에 관한 정리에서, 중심극한정리만큼 중요한 개념

대수의 법칙 예시

- 주사위를 몇 번이고 굴리면 6이 나올 확률은 $\frac{1}{6}$ 에 가까워진다
- 이것을 수학적으로 증명한 것이 대수의 법칙이다

[Python] 대수의 법칙 (10만번 무작위 추출을 4번 진행)

- 주사위가 6이 나오는지 여부로 파악 각 시행은 $Bern(\frac{1}{6})$ 를 따르므로, 표본 크기가 커짐에 따라 표본평균은 $\frac{1}{6}$ 로 수렴하게 된다

```

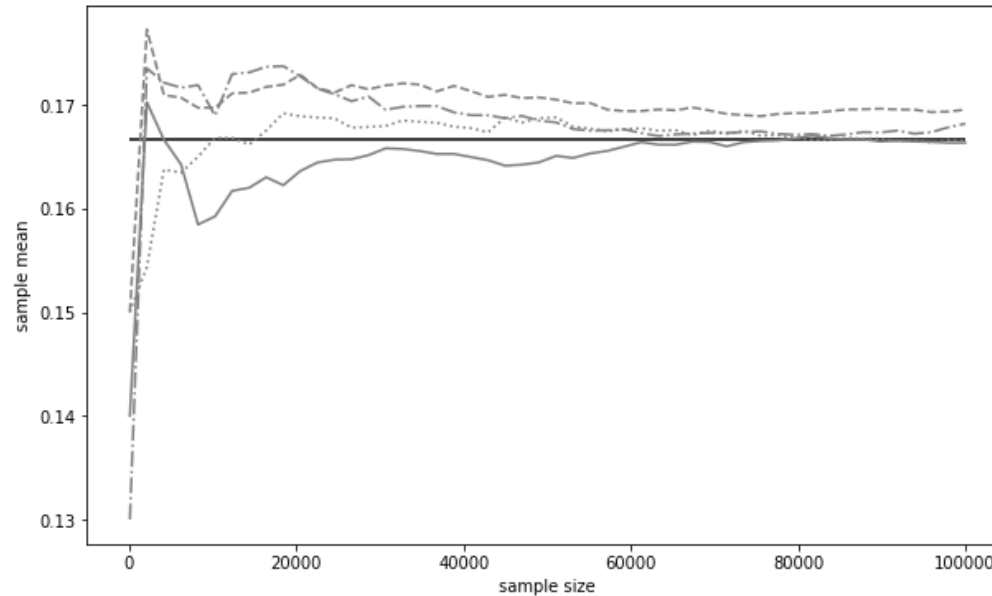
1 p = 1/6
2 rv = stats.bernoulli(p)
3
4 n = int(1e5)
5 sample = rv.rvs((n, 4))
6 space = np.linspace(100, n, 50).astype(int)
7 plot_list = np.array([np.mean(sample[:,sp], axis=0)
8                       for sp in space]).T

```

```

1 fig = plt.figure(figsize=(10, 6))
2 ax = fig.add_subplot(111)
3
4 for pl, ls in zip(plot_list, linestyle):
5     ax.plot(space, pl, ls=ls, color='gray')
6 ax.hlines(p, -1, n, 'k')
7 ax.set_xlabel('sample size')
8 ax.set_ylabel('sample mean')
9
10 plt.show()

```





점 추정 & 구간 추정

1-1 점추정 - 모평균



점 추정 & 구간 추정

1-1. 점추정 - 모평균

.

본격적인 추측통계의 시작

- 추측통계

- > 추정(이번 Chapter) + 검정(다음 Chapter)

- > 점 추정 + 구간 추정

- 점 추정 : 추정하고 싶은 모평균 혹은 모분산이라는 모수를 하나의 수치로 측정
 - 구간 추정 : 폭(구간)으로 모수를 추정하는 방법



파이썬 기초 라이브러리 세팅

- Numpy (분포 구현)
- Pandas (분포 구현)
- Matplotlib의 pyplot 모듈 (시각화)
- Scipy의 stats 모듈 (통계)

```
1  import numpy as np
2  import pandas as pd
3  import matplotlib.pyplot as plt
4  from scipy import stats
5
6  %precision 3
7  %matplotlib inline
```



파이썬 Scikit-learn 기반 기초 데이터 세팅 (RM: 가구당 평균 방 수)

7.1.1. Boston house prices dataset

Data Set Characteristics:

Number of Instances:	506
Number of Attributes:	13 numeric/categorical predictive. Median Value (attribute 14) is usually the target.
Attribute Information (in order):	• CRIM per capita crime rate by town • ZN proportion of residential land zoned for lots over 25,000 sq.ft. • INDUS proportion of non-retail business acres per town • CHAS Charles River dummy variable (= 1 if tract bounds river; 0 otherwise) • NOX nitric oxides concentration (parts per 10 million) • <u>RM average number of rooms per dwelling</u> • AGE proportion of owner-occupied units built prior to 1940 • DIS weighted distances to five Boston employment centres • RAD index of accessibility to radial highways • TAX full-value property-tax rate per \$10,000 • PTRATIO pupil-teacher ratio by town • B 1000(Bk - 0.63)^2 where Bk is the proportion of black people by town • LSTAT % lower status of the population • MEDV Median value of owner-occupied homes in \$1000's

```
1 from sklearn.datasets import load_boston
2
3 boston_houseprice_data = load_boston()
4
5 df = pd.DataFrame(
6     data = boston_houseprice_data.data,
7     columns = boston_houseprice_data.feature_names
8 )
9 rooms = np.array(df[ 'RM' ])
```



점 추정 & 구간 추정

1-1. 점추정 - 모평균

.

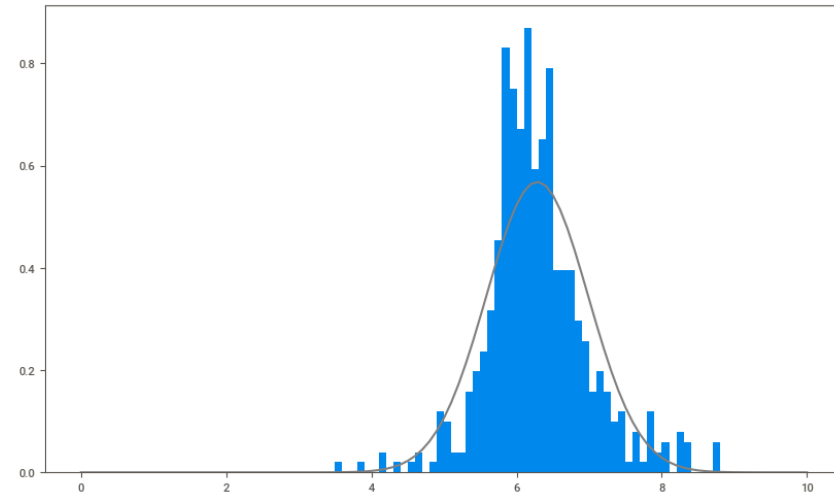
[Python] 기초 데이터의 모평균 μ , 모분산 σ^2 계산

```
1 p_mean = np.mean(rooms)
2 p_var = np.var(rooms)
3
4 p_mean, p_var

(6.285, 0.493)
```

[Python] 기초 데이터 분포와 정규분포 $N(6.285, 0.493^2)$ 의 유사성 확인

```
1 fig = plt.figure(figsize=(10, 6))
2 ax = fig.add_subplot(111)
3
4 xs = np.arange(101)/10
5 rv = stats.norm(p_mean, np.sqrt(p_var))
6 ax.plot(xs, rv.pdf(xs), color='gray')
7 ax.hist(rooms, bins=100, range=(0, 10), density=True)
8
9 plt.show()
```





[Python] 기초 데이터에서 샘플 20개 뽑기 + 20개를 10,000번 뽑기

```
1  np.random.seed(0)
2  n = 20
3  sample = np.random.choice(rooms, n)
4  sample

array([5.572, 6.03 , 6.021, 7.178, 5.708, 6.438, 7.875, 6.112, 6.004,
       5.404, 6.826, 6.358, 6.63 , 6.121, 6.417, 6.437, 7.007, 6.405,
       6.567, 6.8  ])
```

```
1  np.random.seed(1111)
2  n_samples = 10000
3  samples = np.random.choice(rooms, (n_samples, n))
```

- 모평균 및 모분산을 추정하기 위한 데이터로 활용할 예정



모평균의 점추정

- 무작위추출로 얻은 20개의 건물의 가구당 평균 방의 수 샘플은 각 샘플마다 기대값이 μ , 분산이 σ^2 인 확률분포를 따르고, 서로 독립인 확률변수 $X_1, X_2, \dots, X_{20}$ 라 할 수 있다
- 이후 표본평균 $\bar{X} = \frac{X_1 + X_2 + \dots + X_{20}}{20}$ 도 확률변수가 된다
(시행할 때마다 얻는 결과값은 달라진다)
- 최종적으로 표본평균 $\bar{X}$ 의 기대값은 $E(\bar{X}) = \mu$, 모평균이 된다
 - 이렇게 추정량의 기대값이 추측하려는 모수가 되는 성질 = '불편성(unbiasedness)'
 - * 불편성: 편향되지(치우치지) 않음을 뜻함
 - 불편성을 가진 추정량 = '불편추정량(unbiased estimator)'



[Python] 모평균의 점추정 표본평균 및 표본평균의 기대값

1. 20개 표본의 5개 표본평균
2. 20개 표본의 10,000개 표본평균들의 평균
3. 1,000,000개 표본의 1개 표본평균
* 실제 모평균값인 6.285와 같아짐

- 일치성(Consistency)
: 표본크기를 증가시키면 추측하려는 모수에 수렴해가는 성질
- 일치추정량 (Consistent Estimator)
: 일치성을 지닌 추정량

```
1 for i in range(5):  
2     s_mean = np.mean(samples[i])  
3     print(f'{i+1}번째 표본평균: {s_mean:.3f}')
```

```
1번째 표본평균: 6.167  
2번째 표본평균: 6.224  
3번째 표본평균: 6.510  
4번째 표본평균: 6.328  
5번째 표본평균: 6.491
```

```
1 sample_means = np.mean(samples, axis=1)  
2 np.mean(sample_means)
```

```
6.284
```

```
1 np.mean(np.random.choice(rooms, int(1e6)))
```

```
6.285
```



점 추정 & 구간 추정

1-1. 점추정 - 모평균

.

불편성과 일치성을 지닌 추정량

- 불편성 : 추정량의 기대값이 추측하려는 모수가 되는 성질
- 일치성 : 표본크기를 증가시키면 추측하려는 모수에 수렴해가는 성질

즉, 표본평균은 불편성과 일치성 모두를 지닌 모수(모평균)의 '좋은 추정량'이다



점 추정 & 구간 추정

1-2 점추정 - 모분산



모평균의 점추정

- 표본평균은 모수인 모평균을 추정하기위한 좋은 추정량이었다 (불편성 & 일치성)

모분산의 점추정은?

- 먼저, 표본분산 $S^2 = \frac{1}{n} \sum_{i=1}^n (X_i - \bar{X})^2$ 은 표본평균과 동일하게 확률변수로 표현이 되므로, 시행할 때마다 결과가 달라진다.
- 불편성 및 일치성을 가지는지 파악해보도록 하자



[Python] 모분산의 점추정 표본분산 및 표본분산의 기대값

1. 20개 표본의 5개 표본분산
2. 20개 표본의 10,000개 표본분산들의 평균
* 0.467로서 실제 모분산값인 0.493과 크게 다름
즉, 표본분산은 불편추정량이 아님

모분산의 불편추정량은?

- 불편분산 ($\neq$ 표본분산)

- 표본분산 $s^2 = \frac{1}{n} \sum_{i=1}^n (X_i - \bar{X})^2$ 에서 n 으로 나누던 것은 **$n-1$** 로 변경한 것
- 즉, 불편분산은 $s^2 = \frac{1}{n-1} \sum_{i=1}^n (X_i - \bar{X})^2$ 이다

```
1 for i in range(5):
2     s_var = np.var(samples[i])
3     print(f'{i+1}번째 표본분산: {s_var:.3f}')
```

```
1번째 표본분산: 0.874
2번째 표본분산: 0.521
3번째 표본분산: 0.322
4번째 표본분산: 0.221
5번째 표본분산: 0.419
```

```
1 sample_vars = np.var(samples, axis=1)
2 np.mean(sample_vars)
```

```
0.467
```



n-1은 자유도(degree of freedom)라고?

불편분산 표현식 $s^2 = \frac{1}{n-1} \sum_{i=1}^n (X_i - \bar{X})^2$ 에서의 n-1은 자유도를 뜻함

- 자유도 : 자유롭게 값을 취할 수 있는 변수의 개수
 - 표본평균 $\sum_{i=1}^n X_i$ 는 각 X_i 에 아무 제약이 없기에 자유도가 n이다
 - 분산의 경우, $\sum_{i=1}^n (X_i - \bar{X})^2$ 식에서 각 X_i 가 $\frac{1}{n} \sum_{i=1}^n X_i = \bar{X}$ 을 만족해야하는 제약이 있다
따라서, 자유도가 n-1이 된다
- * 자유도로 나누면 왜 불편성을 가지는지는 어려우므로 생략함



[Python] 자유도 $n-1$ 적용된 불편분산 계산 방법 및 1,000,000개 샘플의 불편분산

- var 함수에 ddof (delta degree of freedom) = 1을 지정하면 계산 가능

```
1 sample_u_vars = np.var(samples, axis=1, ddof=1)
2 np.mean(sample_u_vars)

0.491

1 np.var(np.random.choice(rooms, int(1e6)), ddof=1)

0.493
```

- 1,000,000개 샘플의 불편분산 계산시 모분산인 0.493과 거의 같아짐
(일치추정량 만족)



불편성 & 일치성 정리

- 불편성 : 기대값이 추측하기 원하는 모수가 되는 성질
- 일치성 : 표본 크기를 키우면 추측하기 원하는 모수에 수렴하는 성질

모평균과 모분산의 점추정

- $X_1, X_2, \dots, X_n$ 이 서로 독립이고 기대값이 μ , 분산이 σ^2 인 확률분포를 따를 때,
표본평균 $\bar{X}$ 와 불편분산 s^2 은 각각 모평균 μ 와 모분산 σ^2 에 대해
불편성과 일치성을 만족하는 좋은 추정량이다

점 추정 & 구간 추정

2-1 구간추정 - 정규분포의 모평균
(모분산 알 때)



☒ ☒☒ & ☒☒ ☒☒

2-1) 구간추정 – 정규분포의 모평균 (모분산 알 때)

.

구간추정을 배우는 이유

- Ch10. Clip 1에서는 표본평균과 불편분산이 각각 모평균과 모분산의 좋은 추정량임을 배움
- 하지만, 표본평균 및 불편분산은 확률변수이므로, 우연히 편향된 표본을 추출해버리는 경우 예상과 다른 추정이 될 가능성이 있다
- 따라서, 사전에 예상되는 오차를 예측하고, 모평균이 특정 값이 아니라 범위에 있다는 주장을 할 수 있다면 더 안정적인 좋은 추정이 될 것이다



점 추정 & 구간 추정

2-1) 구간추정 - 정규분포의 모평균 (모분산 알 때)

.

구간추정 - 정규분포의 모평균 (모분산 알 때)

- 모집단으로 정규분포를 가정했다면 표본평균 $\bar{X}$ 는 $N(\mu, \frac{\sigma^2}{n})$ 를 따른다
- 추정량의 표준편차인 $\sqrt{\frac{\sigma^2}{n}}$ 를 표준오차(Standard Error)라 한다
- 모분산을 알고 있으므로 표준오차를 계산하는 방법으로, 추정량의 오차를 계산할 수 있다



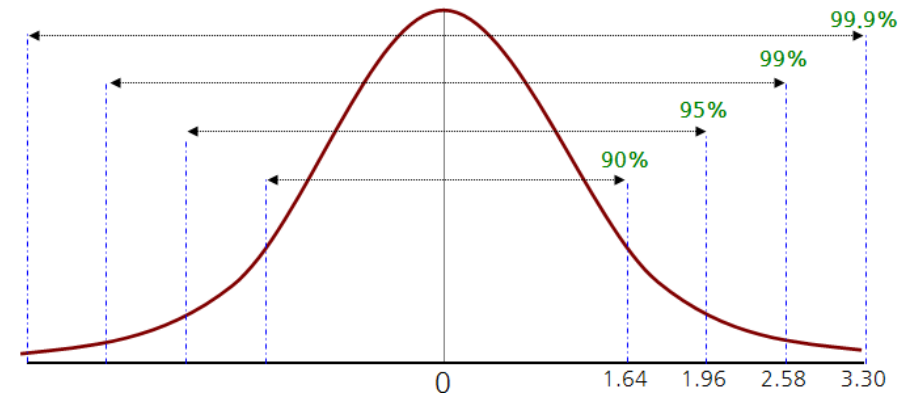
점 추정 & 구간 추정

2-1) 구간추정 - 정규분포의 모평균 (모분산 알 때)

.

구간추정 - 정규분포의 모평균 (모분산 알 때)

- $\bar{X} \sim N(\mu, \frac{\sigma^2}{n})$ 라면 $Z \sim (\bar{X} - \mu) / \sqrt{\frac{\sigma^2}{n}}$ 로 표준화가 가능하다
(표준화를 하면 구간을 계산하기에 유용하다)
- 이후 Z 의 95% 유효구간을 생각해보자



$$P\left(z_{0.975} \leq \frac{\bar{X} - \mu}{\sqrt{\frac{\sigma^2}{n}}} \leq z_{0.025}\right) = 0.95$$

* $z_{0.975}$ 가 약 -1.64, $z_{0.025}$ 가 약 1.64임

구간추정 - 정규분포의 모평균 (모분산 알 때)

- $P\left(z_{0.975} \leq \frac{\bar{X} - \mu}{\sqrt{\frac{\sigma^2}{n}}} \leq z_{0.025}\right) = 0.95$ 이 식을 μ 에 관해 다시 변환하면

$$P\left(\bar{X} - z_{0.025}\sqrt{\frac{\sigma^2}{n}} \leq \mu \leq \bar{X} - z_{0.975}\sqrt{\frac{\sigma^2}{n}}\right) = 0.95 \text{ 이 된다}$$

- 즉, 모평균 μ 를 포함하는 확률이 95%인 구간은 아래와 같다

$$\left[\bar{X} - z_{0.025}\sqrt{\frac{\sigma^2}{n}}, \bar{X} - z_{0.975}\sqrt{\frac{\sigma^2}{n}} \right]$$

- 위와 같이 구간추정으로 구한 구간을 신뢰수준 95%인 신뢰구간(Confidence Interval)이라 표현한다



점 추정 & 구간 추정

2-1) 구간추정 - 정규분포의 모평균 (모분산 알 때)

.

구간추정 - 모분산 알 때 정규분포의 모평균의 신뢰구간 일반화

- $X_1, X_2, \dots, X_n$ 이 서로 독립이고 기대값이 μ , 분산이 σ^2 인 확률분포를 따를 때, 모분산 σ^2 을 알고 있다면, 신뢰수준 $100(1-\alpha)\%$ 인 신뢰구간은 아래와 같다

$$\left[\bar{X} - z_{\alpha/2} \sqrt{\frac{\sigma^2}{n}}, \bar{X} - z_{1-\alpha/2} \sqrt{\frac{\sigma^2}{n}} \right]$$



점 추정 & 구간 추정

2-1) 구간추정 - 정규분포의 모평균 (모분산 알 때)

.

[Python] 표본 데이터로 모평균의 95% 신뢰구간 계산

```
1  rv = stats.norm()  
2  lcl = s_mean - rv.isf(0.025) * np.sqrt(p_var/n)  
3  ucl = s_mean + rv.isf(0.975) * np.sqrt(p_var/n)  
4  
5  lcl, ucl  
  
(6.088, 6.703)
```

- 모평균 6.285가 구간내에 포함되어있음을 확인



점 추정 & 구간 추정

2-1) 구간추정 – 정규분포의 모평균 (모분산 알 때)

.

신뢰구간 해석시 범하기 쉬운 오류

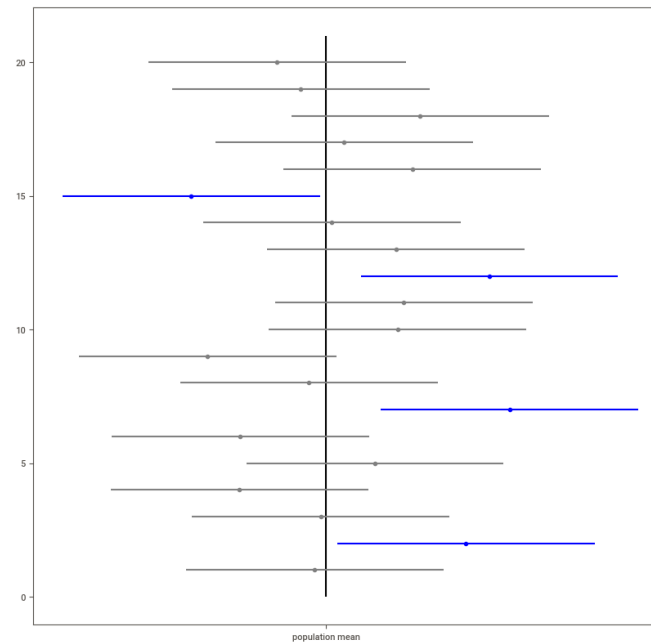
- 95% 신뢰구간이 [6.088, 6.703]이라면, 모평균이 95% 확률로 위 범위 안에 들어간다?
 - **No, 그렇지 않다**
 - * 모평균 신뢰구간 추정을 100번하면 모평균이 95번 정도의 비율로 모평균이 포함될 것이라는 뜻이기 때문
- 옳은 해석
 - : 동일한 방법으로 수차례 표본추출하여 구간추정시 95%의 구간추정에는 모평균이 포함되어 있다

[Python] 신뢰구간 해석을 위한 20회 추정을 통한 시각화 제시

```

1  fig = plt.figure(figsize=(10, 10))
2  ax = fig.add_subplot(111)
3
4  rv = stats.norm()
5  n_samples = 20
6  ax.vlines(p_mean, 0, 21)
7  for i in range(n_samples):
8      sample_ = samples[i]
9      s_mean_ = np.mean(sample_)
10     lcl = s_mean_ - rv.isf(0.025) * np.sqrt(p_var/n)
11     ucl = s_mean_ - rv.isf(0.975) * np.sqrt(p_var/n)
12     if lcl <= p_mean <= ucl:
13         ax.scatter(s_mean_, n_samples-i, color='gray')
14         ax.hlines(n_samples-i, lcl, ucl, color='gray')
15     else:
16         ax.scatter(s_mean_, n_samples-i, color='b')
17         ax.hlines(n_samples-i, lcl, ucl, color='b')
18 ax.set_xticks([p_mean])
19 ax.set_xticklabels(['population mean'])
20
21 plt.show()

```



- 즉, 총 4회가 모평균을 포함하지 않은 구간추정임을 알 수 있다
(95% 신뢰구간으로 예측하였지만, 80%의 추정이 모평균을 포함하고 있음을 확인)



점 추정 & 구간 추정

2-1) 구간추정 - 정규분포의 모평균 (모분산 알 때)

.

[Python] 신뢰구간 해석을 위한 10,000회 추정시 모평균을 포함할 확률 계산

```
1  rv = stats.norm()
2  cnt = 0
3
4  for sample_ in samples:
5      s_mean_ = np.mean(sample_)
6      lcl = s_mean_ - rv.isf(0.025) * np.sqrt(p_var/n)
7      ucl = s_mean_ + rv.isf(0.975) * np.sqrt(p_var/n)
8      if lcl <= p_mean <= ucl:
9          cnt += 1
10 cnt / len(samples)
```

0.947

- 10,000개 신뢰구간의 약 94.7%가 모평균을 포함하고 있음을 확인



점 추정 & 구간 추정

2-2) 구간추정 - 정규분포의 모분산



정규분포의 모분산 구간추정 (모평균 모를 때)

- 모분산 구간추정의 목표는 $P(* \leq \sigma^2 \leq *) = 0.95$ 를 만족하는 구간을 구하는 것!

정규분포의 불편분산 s^2 과 관련된 확률분포? 카이제곱 분포

- 표본평균 $\bar{X} \sim N(\mu, \frac{\sigma^2}{n})$ 를 $Z \sim (\bar{X} - \mu) / \sqrt{\frac{\sigma^2}{n}}$ 로 변환하였듯이,
불편분산 s^2 또한 대표적인 확률분포의 형태로 변환할 필요가 있음
- **카이제곱 분포로의 변환**
 - 불편분산 s^2 에 $Y = \frac{(n-1)s^2}{\sigma^2}$ 이라는 변환을 하면 $Y \sim \chi^2(n-1)$ 이 된다고 함

[Python] 10,000개 표본 데이터 기반 Y의 표본 데이터 생성

```

1 sample_y = sample_u_vars * (n-1) / p_var
2 sample_y

array([35.474, 21.152, 13.053, ..., 17.608, 24.126,  9.3  ])

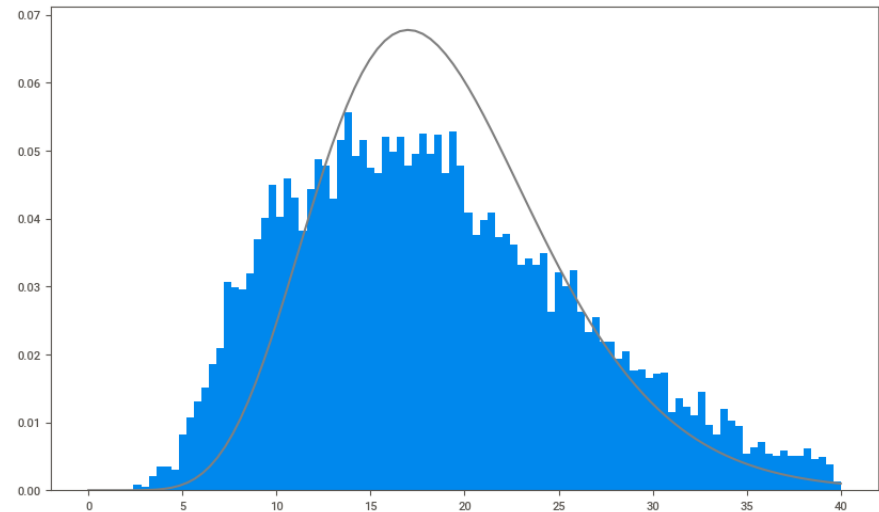
```

[Python] Y의 표본데이터와 $\chi^2(n-1)$ 시각화

```

1 fig = plt.figure(figsize=(10, 6))
2 ax = fig.add_subplot(111)
3
4 xs = np.linspace(0, 40, 100)
5 rv = stats.chi2(df=n-1)
6 ax.plot(xs, rv.pdf(xs), color='gray')
7 hist, _, _ = ax.hist(sample_y, bins=100,
8 | | | | | range=(0, 40), density=True)
9
10 plt.show()

```





정규분포의 모분산 구간추정 (모평균 모를 때)

- 모분산의 95% 신뢰구간 계산을 위해 카이제곱 분포에 의해서 먼저 정의하고

$$P\left(\chi_{0.975}^2(n-1) \leq \frac{(n-1)s^2}{\sigma^2} \leq \chi_{0.025}^2(n-1)\right) = 0.95$$

위 식을 σ^2 에 관해 다시 정리하면 다음과 같다

$$P\left(\frac{(n-1)s^2}{\chi_{0.025}^2(n-1)} \leq \sigma^2 \leq \frac{(n-1)s^2}{\chi_{0.975}^2(n-1)}\right) = 0.95$$

즉, 모분산 σ^2 를 포함하는 확률이 95%인 구간은 아래와 같다

$$\left[\frac{(n-1)s^2}{\chi_{0.025}^2(n-1)}, \frac{(n-1)s^2}{\chi_{0.975}^2(n-1)} \right]$$



구간추정 - 모평균 모를 때 정규분포 모분산의 신뢰구간 일반화

- $X_1, X_2, \dots, X_n$ 이 서로 독립이고 기대값이 μ , 분산이 σ^2 인 확률분포를 따를 때, 모평균 μ 을 모르고 있다면, 신뢰수준 $100(1-\alpha)\%$ 인 신뢰구간은 아래와 같이 추정된다

$$\left[\frac{(n-1)s^2}{\chi_{\alpha/2}^2(n-1)}, \frac{(n-1)s^2}{\chi_{1-\alpha/2}^2(n-1)} \right]$$



점 추정 & 구간 추정

2-2) 구간추정 - 정규분포의 모분산

.

[Python] 표본 데이터로 모분산의 95% 신뢰구간 계산

```
1  rv = stats.chi2(df=n-1)
2  lcl = (n-1) * u_var / rv.isf(0.025)
3  hcl = (n-1) * u_var / rv.isf(0.975)
4
5  lcl, hcl

(0.193, 0.711)
```

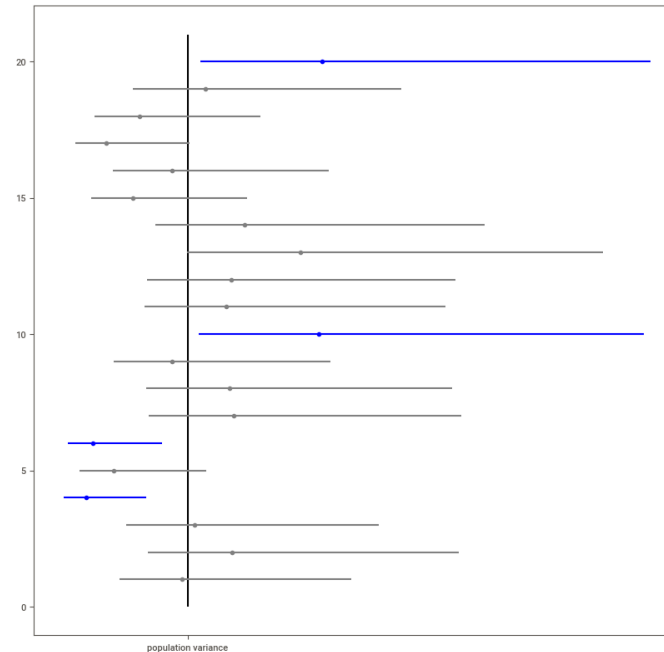
- 모분산 0.493이 구간내에 포함되어있음을 확인

[Python] 신뢰구간 해석을 위한 20회 추정을 통한 시각화 제시

```

1  fig = plt.figure(figsize=(10, 10))
2  ax = fig.add_subplot(111)
3
4  rv = stats.chi2(df=n-1)
5  n_samples = 20
6  ax.vlines(p_var, 0, 21)
7  for i in range(n_samples):
8      sample_ = samples[i]
9      u_var_ = np.var(sample_, ddof=1)
10     lcl = (n-1) * u_var_ / rv.isf(0.025)
11     ucl = (n-1) * u_var_ / rv.isf(0.975)
12     if lcl <= p_var <= ucl:
13         ax.scatter(u_var_, n_samples-i, color='gray')
14         ax.hlines(n_samples-i, lcl, ucl, 'gray')
15     else:
16         ax.scatter(u_var_, n_samples-i, color='b')
17         ax.hlines(n_samples-i, lcl, ucl, 'b')
18 ax.set_xticks([p_var])
19 ax.set_xticklabels(['population variance'])
20
21 plt.show()

```



- 즉, 총 4회가 모분산을 포함하지 않은 구간추정임을 알 수 있다
(95% 신뢰구간으로 예측하였지만, 80%의 추정이 모분산을 포함하고 있음을 확인)



점 추정 & 구간 추정

2-2) 구간추정 - 정규분포의 모분산

.

[Python] 신뢰구간 해석을 위한 10,000회 추정시 모평균을 포함할 확률 계산

```
1  rv = stats.chi2(df=n-1)
2  cnt = 0
3  for sample_ in samples:
4      u_var_ = np.var(sample_, ddof=1)
5      lcl = (n-1) * u_var_ / rv.isf(0.025)
6      ucl = (n-1) * u_var_ / rv.isf(0.975)
7      if lcl <= p_var <= ucl:
8          cnt += 1
9
10 cnt / len(samples)
```

0.845

- 10,000개 신뢰구간의 약 84.5%가 모분산을 포함하고 있음을 확인 (95%와 약간의 차이가 있음을 확인)

점 추정 & 구간 추정

2-3) 구간추정 - 정규분포의 모평균
(모분산 모를 때)

모분산 모를 때, 정규분포 모평균의 구간추정

- 실제로는 모평균 뿐만아니라 모분산 또한 모르는 경우가 대부분이다
- 모분산 기반 표준오차 $\sqrt{\frac{\sigma^2}{n}}$ 가 아니더라도 간단히 모분산 대신 불편분산을 기반으로 한 $\sqrt{\frac{s^2}{n}}$ 을 표준오차로 대신 사용하면 된다

즉, $t = (\bar{X} - \mu) / \sqrt{\frac{s^2}{n}}$ 로 변환이 가능하다

* 이는 실제 표준오차를 사용하지 않기 때문에, t는 표준정규분포를 따르지 않고, Chapter 8에서 정의한 자유도가 n-1인 t 분포를 따르게 된다



모분산 모를 때 정규분포 모평균의 구간추정

- $P\left(t_{0.975}(n-1) \leq \frac{\bar{X}-\mu}{\sqrt{\frac{s^2}{n}}} \leq t_{0.025}(n-1)\right) = 0.95$ 이 식을 μ 에 관해 다시 변환하면

$$P\left(\bar{X} - t_{0.025}(n-1)\sqrt{\frac{s^2}{n}} \leq \mu \leq \bar{X} - t_{0.975}(n-1)\sqrt{\frac{s^2}{n}}\right) = 0.95 \text{ 이 된다}$$

- 즉, 모평균 μ 를 포함하는 확률이 95%인 구간은 아래와 같다

$$\left[\bar{X} - t_{0.025}(n-1)\sqrt{\frac{s^2}{n}}, \bar{X} - t_{0.975}(n-1)\sqrt{\frac{s^2}{n}} \right]$$



점 추정 & 구간 추정

2-3) 구간추정 - 정규분포의 모평균 (모분산 모를 때)

.

모분산 모를 때 정규분포 모평균의 신뢰구간 일반화

- $X_1, X_2, \dots, X_n$ 이 서로 독립이고 기대값이 μ , 분산이 σ^2 인 확률분포를 따를 때, 모분산 σ^2 을 모른다면, 신뢰수준 $100(1-\alpha)\%$ 인 신뢰구간은 아래와 같다

$$\left[\bar{X} - t_{\frac{\alpha}{2}}(n-1) \sqrt{\frac{s^2}{n}}, \bar{X} - t_{1-\frac{\alpha}{2}}(n-1) \sqrt{\frac{s^2}{n}} \right]$$



점 추정 & 구간 추정

2-3) 구간추정 - 정규분포의 모평균 (모분산 모를 때)

.

[Python] 표본 데이터로 모평균의 95% 신뢰구간 계산

```
1  rv = stats.t(df=n-1)
2  lcl = s_mean - rv.isf(0.025) * np.sqrt(u_var/n)
3  ucl = s_mean + rv.isf(0.975) * np.sqrt(u_var/n)
4
5  lcl, ucl

(6.125, 6.666)
```

- 모평균 6.285가 구간내에 포함되어있음을 확인

점 추정 & 구간 추정

2-4) 구간추정 & 신뢰구간 예
(베르누이 분포 & 푸아송 분포)

모분산 모를 때 베르누이 분포 모평균의 구간추정

- 베르누이 분포의 표본평균 $\bar{X}$ 는 중심극한정리에 따라 $N(p, \frac{p(1-p)}{n})$ 을 따른다

- $P\left(z_{0.975} \leq \frac{\bar{X}-p}{\sqrt{\frac{p(1-p)}{n}}} \leq z_{0.025}\right) = 0.95$ 이 식을 p 에 관해 다시 변환하면

$$P\left(\bar{X} - z_{0.025}\sqrt{\frac{\bar{X}(1-\bar{X})}{n}} \leq p \leq \bar{X} - z_{0.975}\sqrt{\frac{\bar{X}(1-\bar{X})}{n}}\right) = 0.95$$

이 식에서 $p(1-p)$ 를 $\bar{X}(1-\bar{X})$ 로 다시 변환하면

$$P\left(\bar{X} - z_{0.025}\sqrt{\frac{\bar{X}(1-\bar{X})}{n}} \leq p \leq \bar{X} - z_{0.975}\sqrt{\frac{\bar{X}(1-\bar{X})}{n}}\right) = 0.95$$

이 된다



점 추정 & 구간 추정

2-4) 구간추정 & 신뢰구간 예 (베르누이 분포 & 푸아송 분포)

.

모분산 모를 때 베르누이 분포 모평균의 구간추정 및 일반화

- 즉, 모평균 p 를 포함하는 확률이 95%인 구간은 아래와 같다

$$\left[\bar{X} - z_{0.025} \sqrt{\frac{\bar{X}(1 - \bar{X})}{n}}, \bar{X} - z_{0.975} \sqrt{\frac{\bar{X}(1 - \bar{X})}{n}} \right]$$

- 모평균의 $100(1-\alpha)\%$ 의 신뢰구간은 아래와 같다

$$\left[\bar{X} - z_{\alpha/2} \sqrt{\frac{\bar{X}(1 - \bar{X})}{n}}, \bar{X} - z_{1-\alpha/2} \sqrt{\frac{\bar{X}(1 - \bar{X})}{n}} \right]$$



점 추정 & 구간 추정

2-4) 구간추정 & 신뢰구간 예 (베르누이 분포 & 푸아송 분포)

.

모분산 모를 때 푸아송 분포 모평균의 구간추정

- 푸아송 분포의 표본평균 $\bar{X}$ 는 중심극한정리에 따라 $N(\lambda, \frac{\lambda}{n})$ 을 따른다

- $P\left(z_{0.975} \leq \frac{\bar{X}-\lambda}{\sqrt{\frac{\lambda}{n}}} \leq z_{0.025}\right) = 0.95$ 이 식을 λ 에 관해 다시 변환하면

$$P\left(\bar{X} - z_{0.025}\sqrt{\frac{\bar{X}}{n}} \leq \lambda \leq \bar{X} - z_{0.975}\sqrt{\frac{\bar{X}}{n}}\right) = 0.95 \text{ 이 식에서 } \lambda \text{ 를 } \bar{X} \text{로 다시 변환하면}$$

$$P\left(\bar{X} - z_{0.025}\sqrt{\frac{\bar{X}}{n}} \leq p \leq \bar{X} - z_{0.975}\sqrt{\frac{\bar{X}}{n}}\right) = 0.95 \text{ 이 된다}$$



점 추정 & 구간 추정

2-4) 구간추정 & 신뢰구간 예 (베르누이 분포 & 푸아송 분포)

.

모분산 모를 때 푸아송 분포 모평균의 구간추정 및 일반화

- 즉, 모평균 p 를 포함하는 확률이 95%인 구간은 아래와 같다

$$\left[\bar{X} - z_{0.025} \sqrt{\frac{\bar{X}}{n}}, \bar{X} - z_{0.975} \sqrt{\frac{\bar{X}}{n}} \right]$$

- 모평균의 $100(1-\alpha)\%$ 의 신뢰구간은 아래와 같다

$$\left[\bar{X} - z_{\alpha/2} \sqrt{\frac{\bar{X}}{n}}, \bar{X} - z_{1-\alpha/2} \sqrt{\frac{\bar{X}}{n}} \right]$$

Q&A

질의응답 (5분)





다음 주 배울 내용

Contents for Next Week

- 데이터 분석 예제
- 고급 Numpy
- 가설 검정



끝. 감사합니다.

수업 듣느라 수고하셨습니다.

